

High performance AI operators and operator Engineering

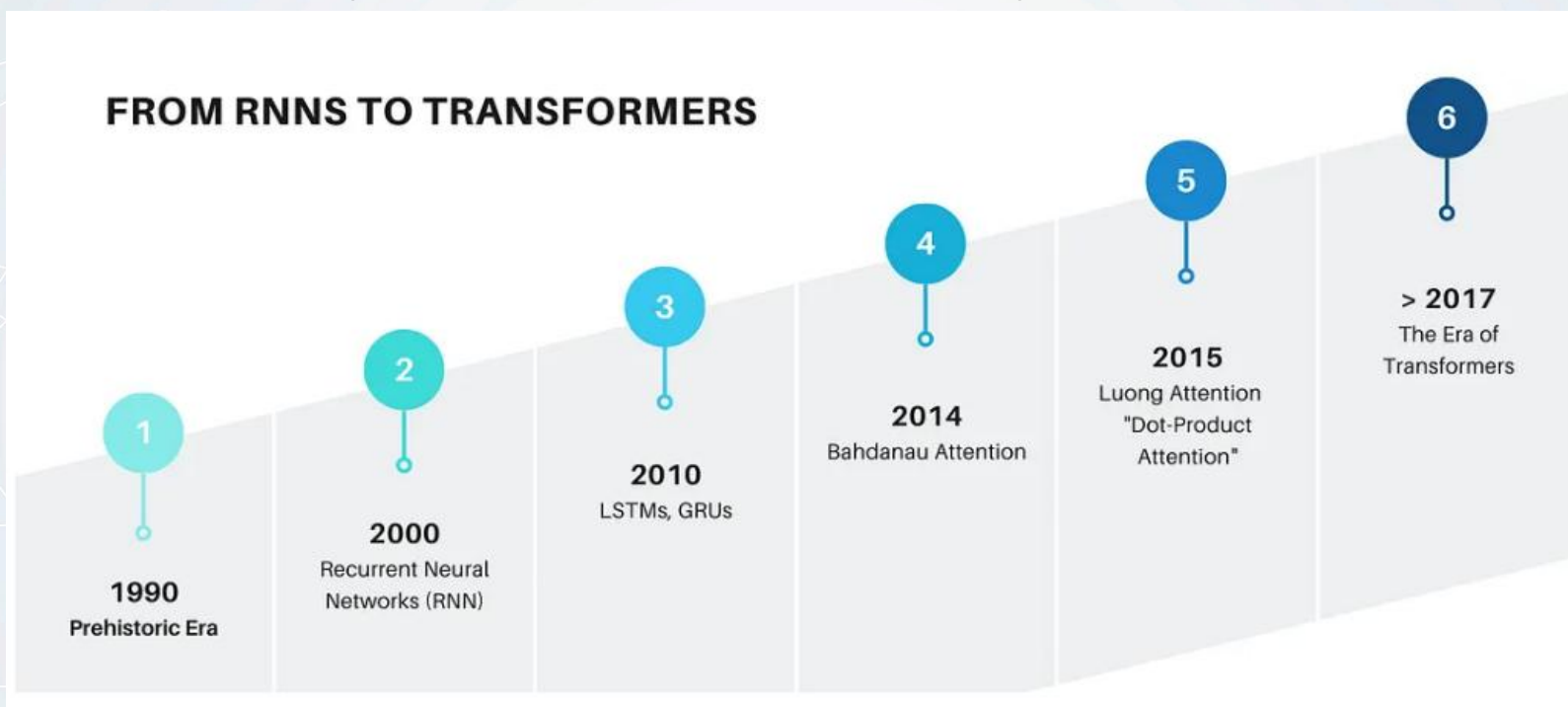
Hongyu Liu 2026.08.05

Outline

- Attention & Flash Attention
- Operator Fusion + Case Study
- Architectural Design of Operator Libraries + Case Study

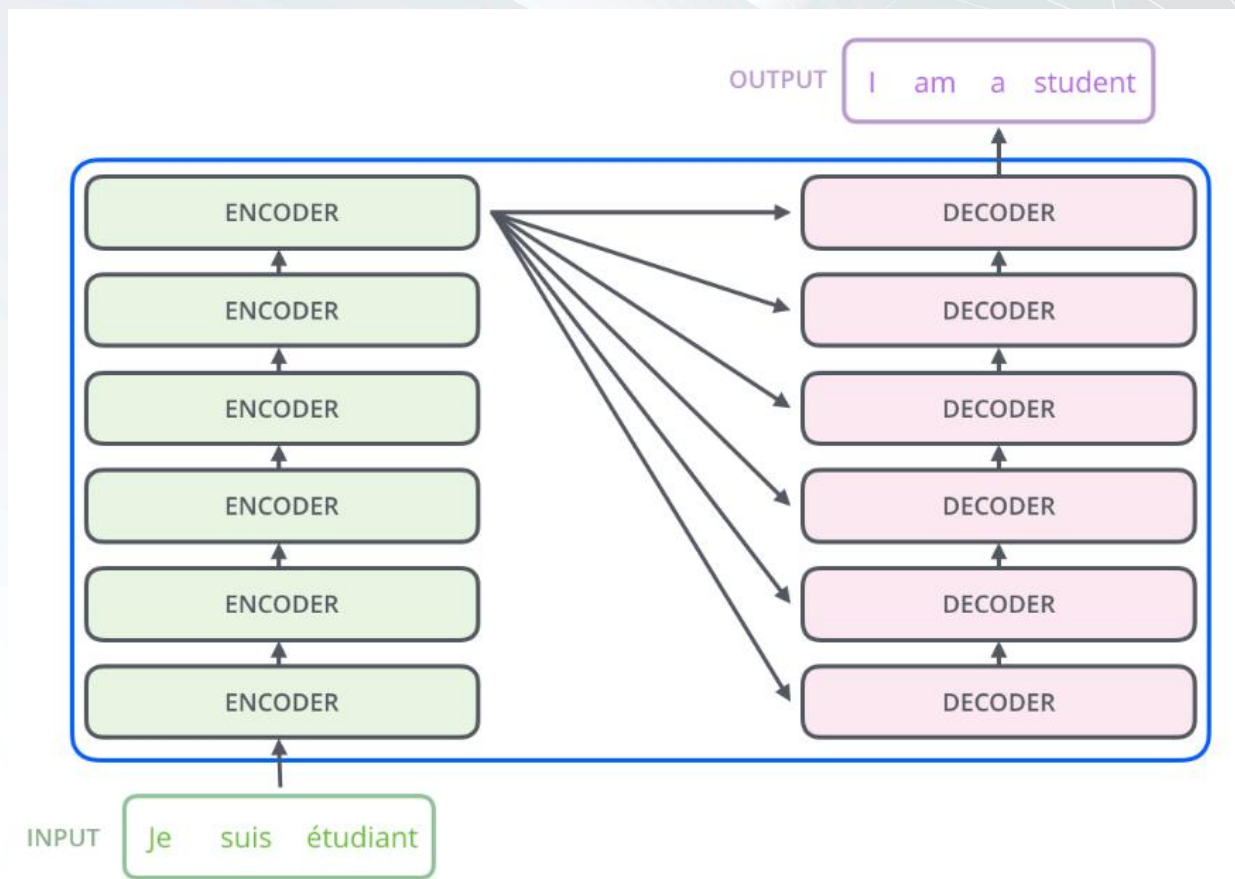
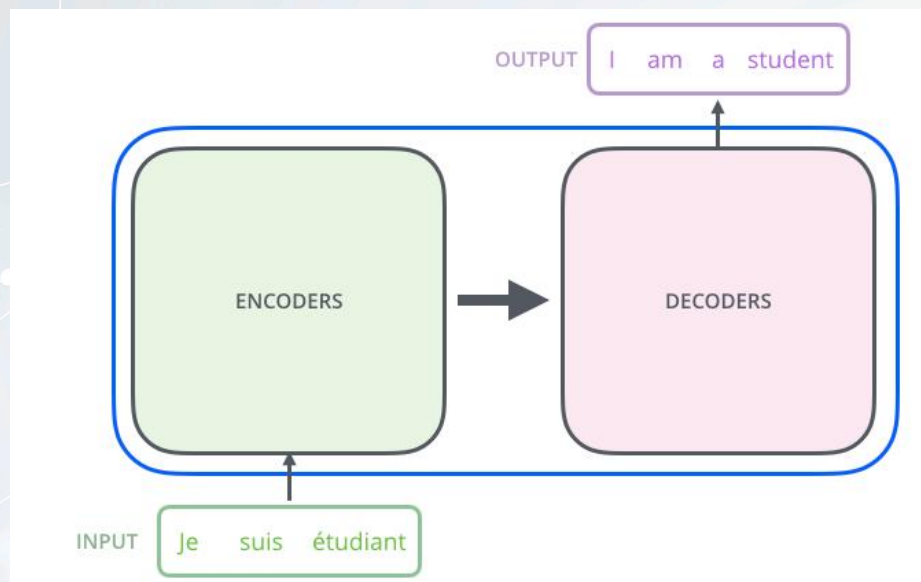
• From RNN to Transformer

- 1980s – RNN (Serial Order: a parallel distributed processing approach)
- 1997 – LSTM (Long Short Term Memory)
- 2014 – GRU (Neural Machine Translation by Jointly Learning to Align and Translate)
- 2017 – Transformer (Attention is All Your Need)



• A Brief Overview of the Transformer Architecture

• A High-Level Look



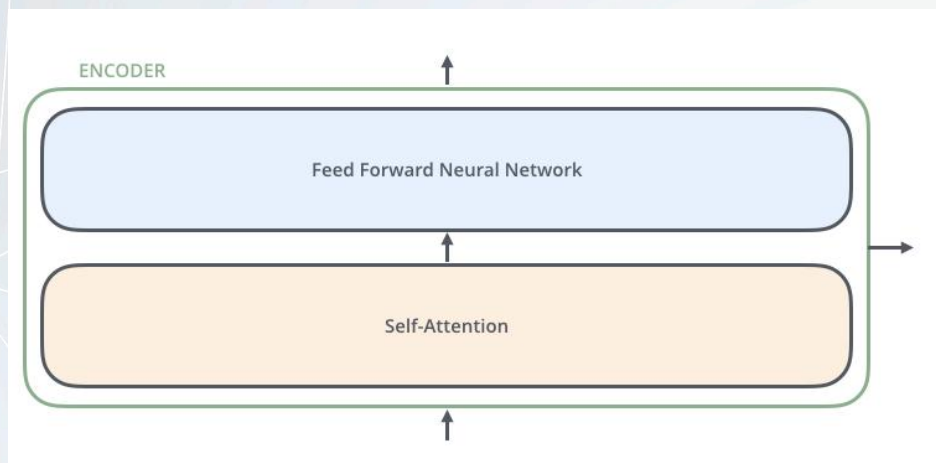
• Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).

• <https://jalammar.github.io/illustrated-transformer/>

• Background: Transformer (1/3)

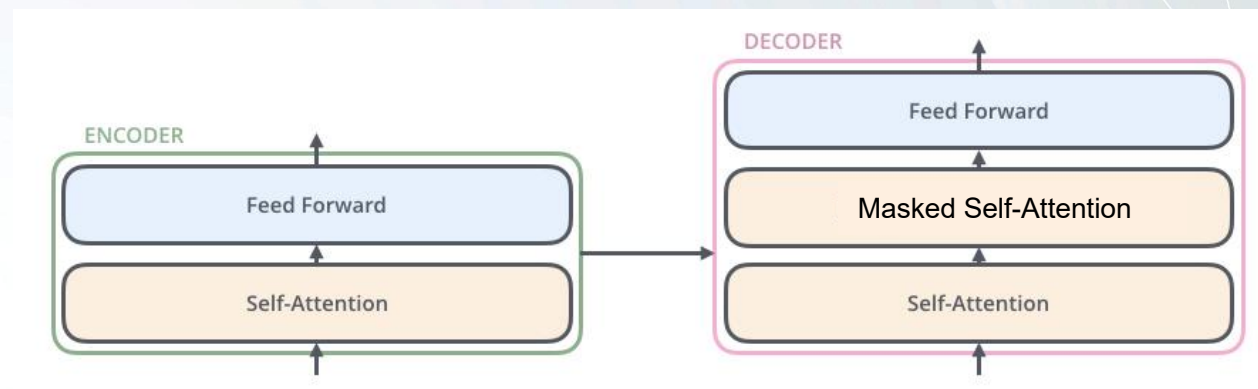
- Encoder

- Multi-Head Self-Attention Mechanism
- Simple, Position-wise Feed-Forward Network



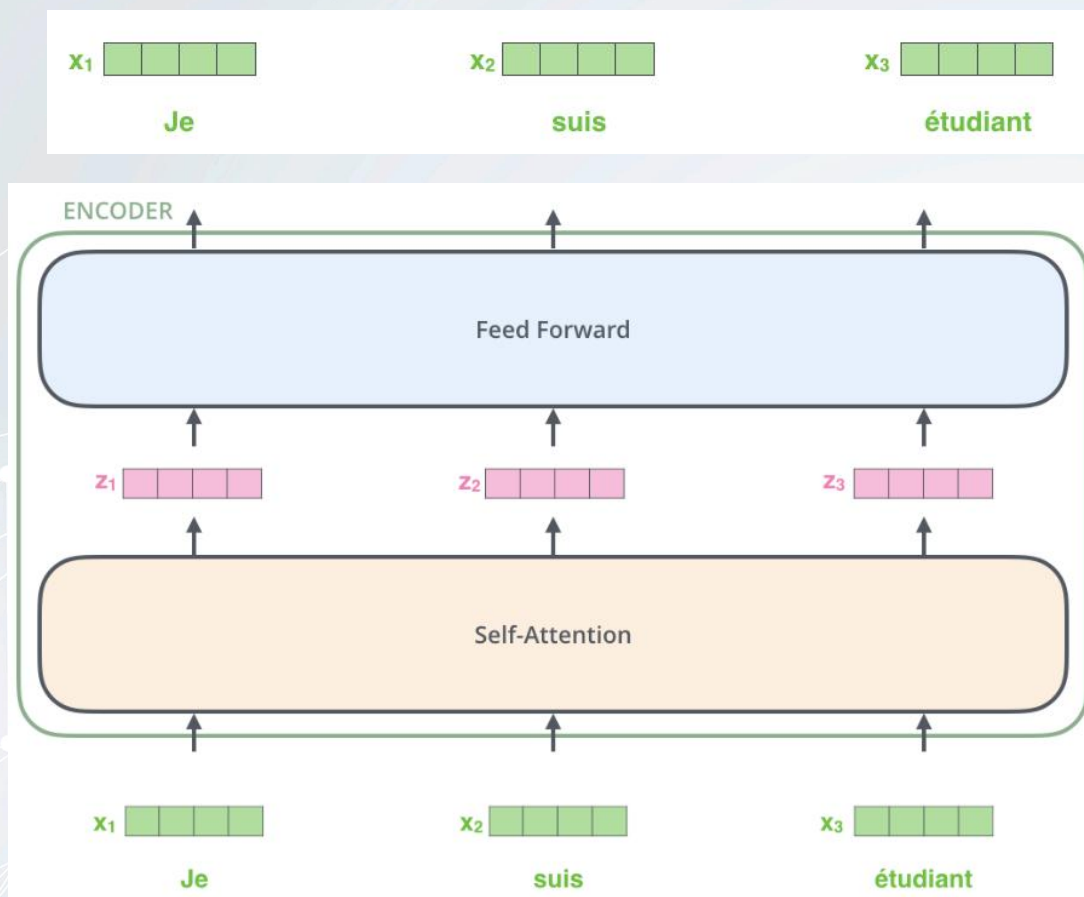
- Decoder

- Masked Self-Attention
- Encoder-Decoder Attention
- Feed-Forward Network



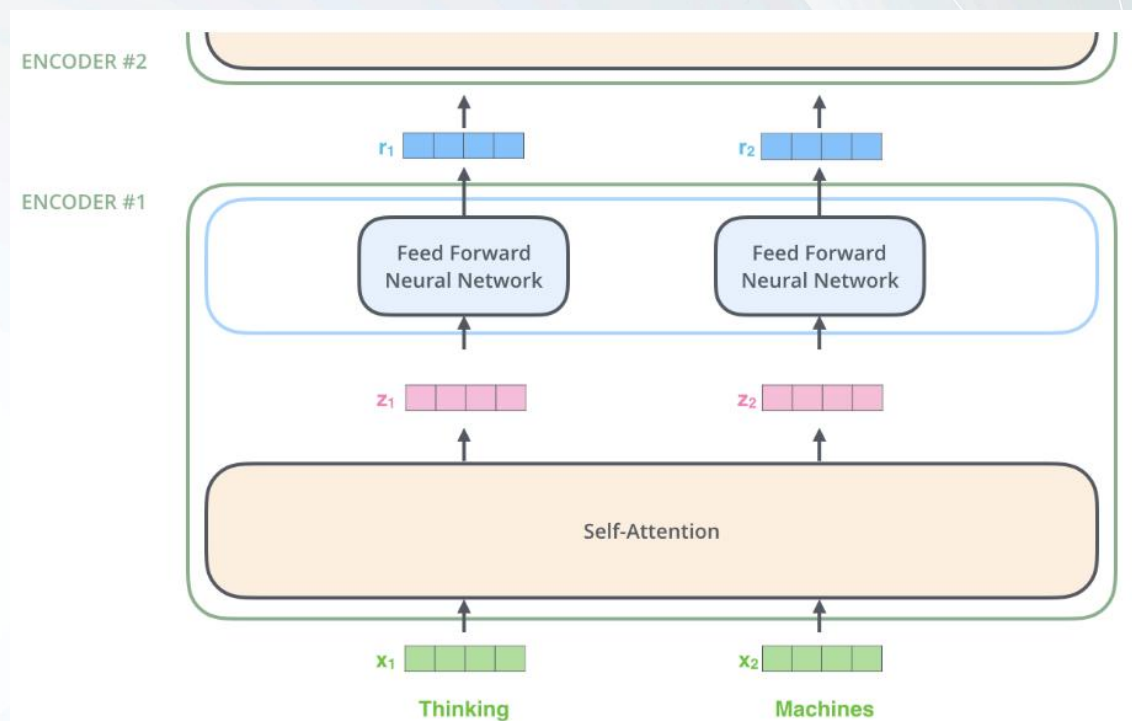
• Background: Transformer (2/3)

• Tensors



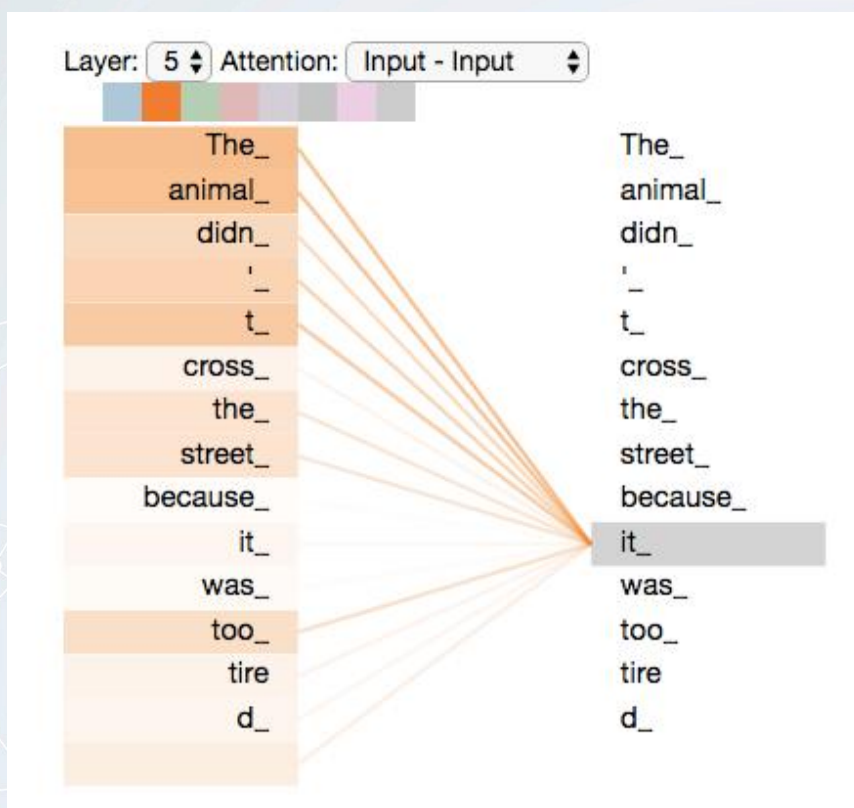
• Encoding Flow

- Dependent paths in self-attention layer
- Interdependent paths in feed-forward layer



• Background: Transformer (3/3)

- Self-Attention at a High Level
 - The animal didn't cross the street because it was too tired



• Q/K/V Matrices (1/2)

- Self-Attention as Information Retrieval
- The Triplets:
 - Each token is transformed into three vectors
 - Query (Q): “What am I looking for?”
 - Key (K): “What information do I contain?”
 - Value (V): “What is the actual content I offer?”
- The Mechanism:
 - Similarity: Compute the compatibility between the Query of the current token and the Keys of all other tokens.
 - Weights: Pass the scores through a Softmax function to get attention weights.
 - Output: The final representation is a weighted sum of the Values.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

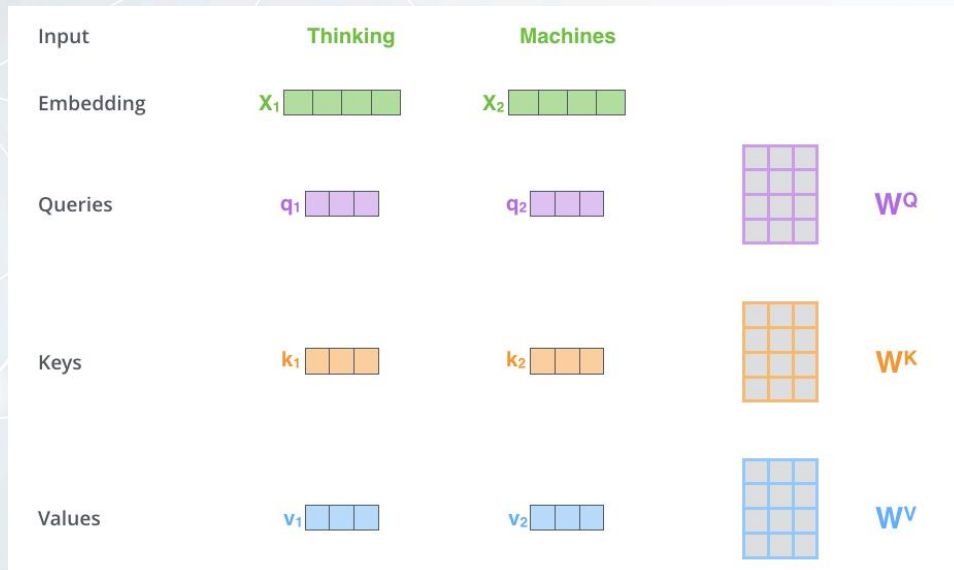
Algorithm 0 Standard Attention Implementation

Require: Matrices $Q, K, V \in \mathbb{R}^{N \times d}$ in HBM.

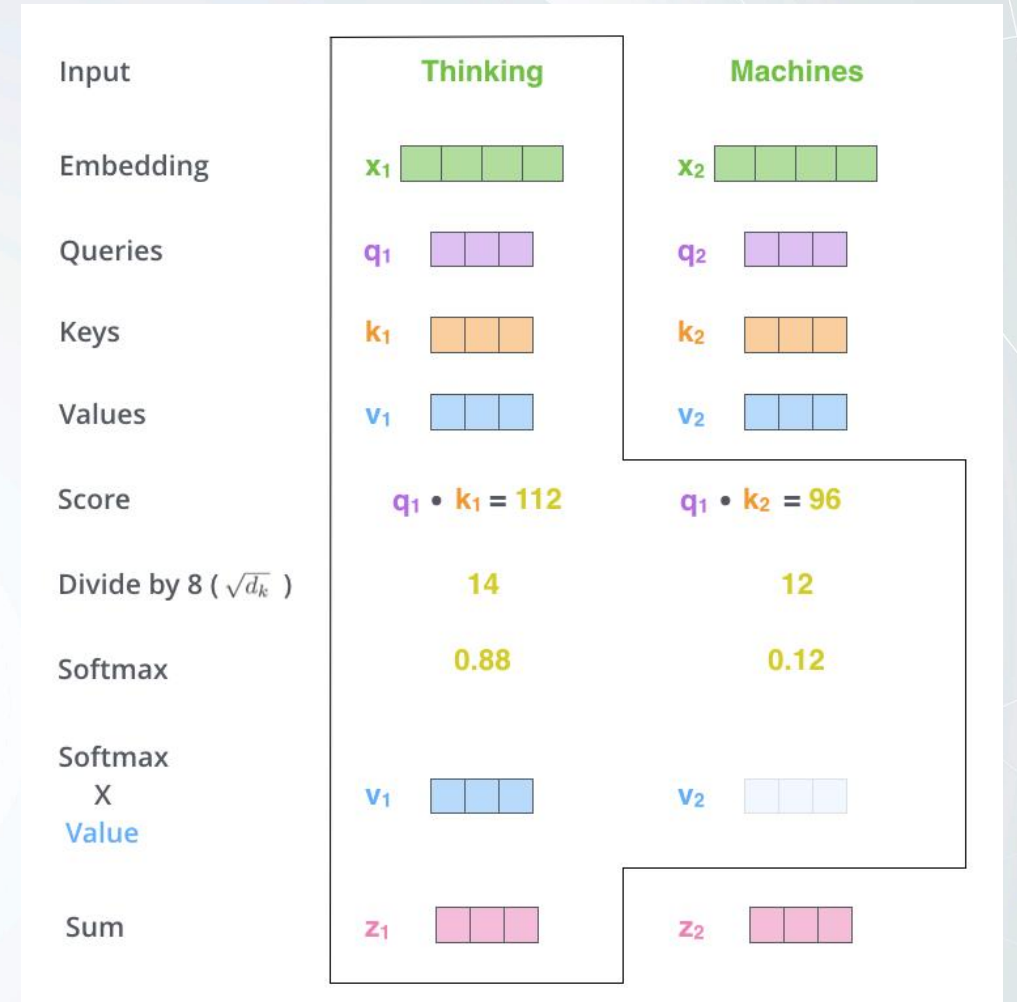
- 1: Load Q, K by blocks from HBM, compute $S = QK^T$, write S to HBM.
 - 2: Read S from HBM, compute $P = \text{softmax}(S)$, write P to HBM.
 - 3: Load P and V by blocks from HBM, compute $O = PV$, write O to HBM.
 - 4: Return O .
-

• Q/K/V Matrices (2/2)

- Create Query, Key and Value
 - Query: The **current token's "question"** to all others about relevance.
 - Key: The **token's identifying information** to answer the "question."
 - Value: The **token's content** to be passed through (weighted) to the output.



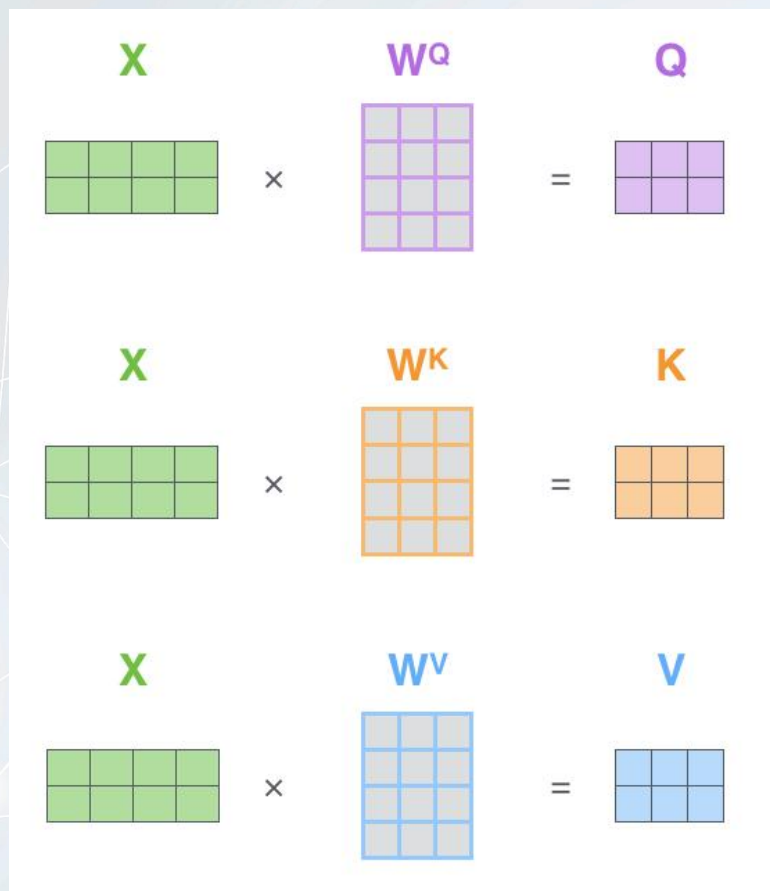
- Score and Weight



• Computation Pipeline of $Attention(Q, K, V)$



- Calculate Query, Key, and Value matrices
- Score and Weight



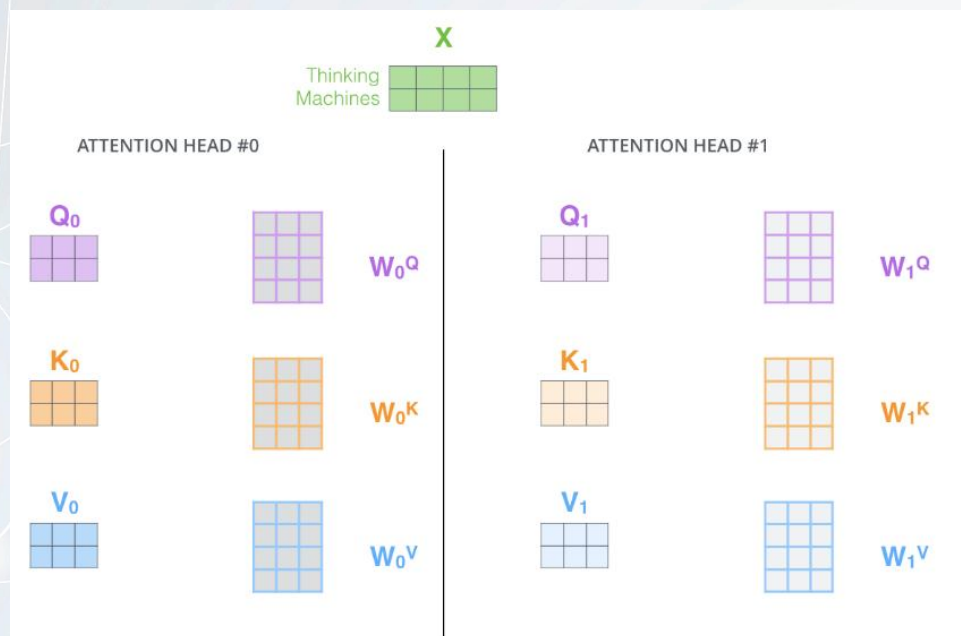
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

= Z (pink 2x3 grid)

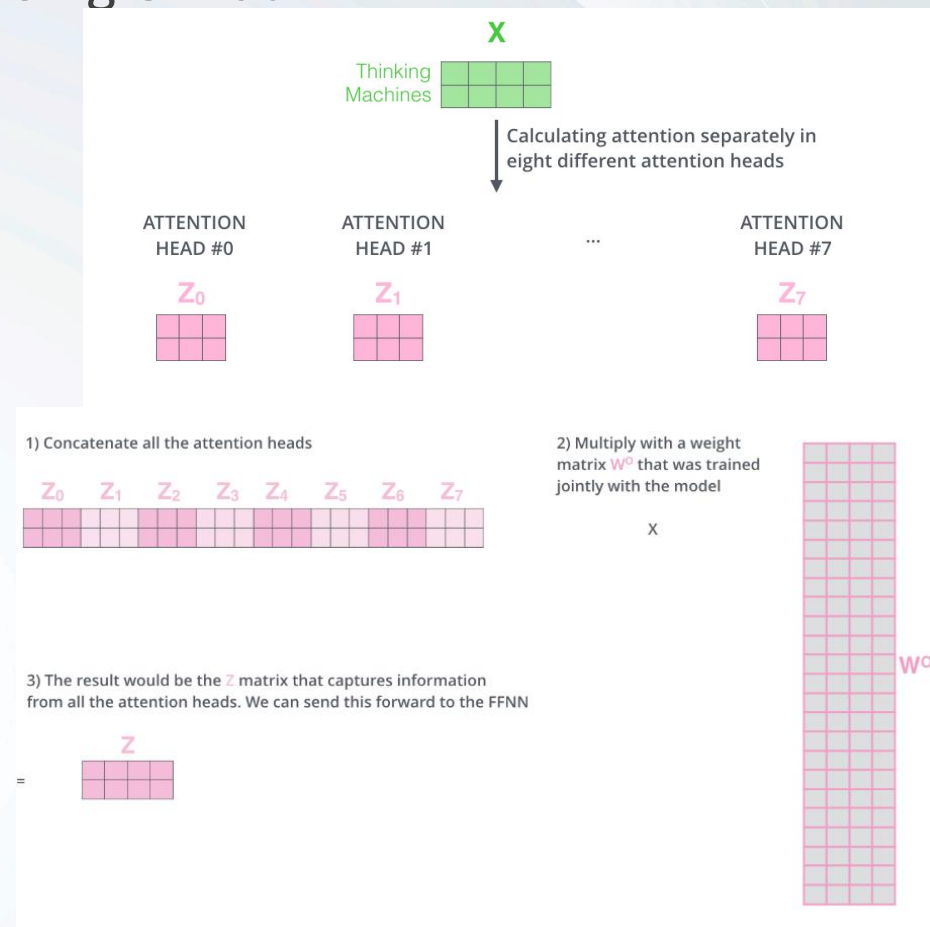
Transformer: Multi-Headed Attention (1/2)

Multi-Headed Attention

- Expands the model's ability to focus on different positions
- Gives the attention layer multiple "representation subspaces"



- Condense these matrixs down into a single matrix



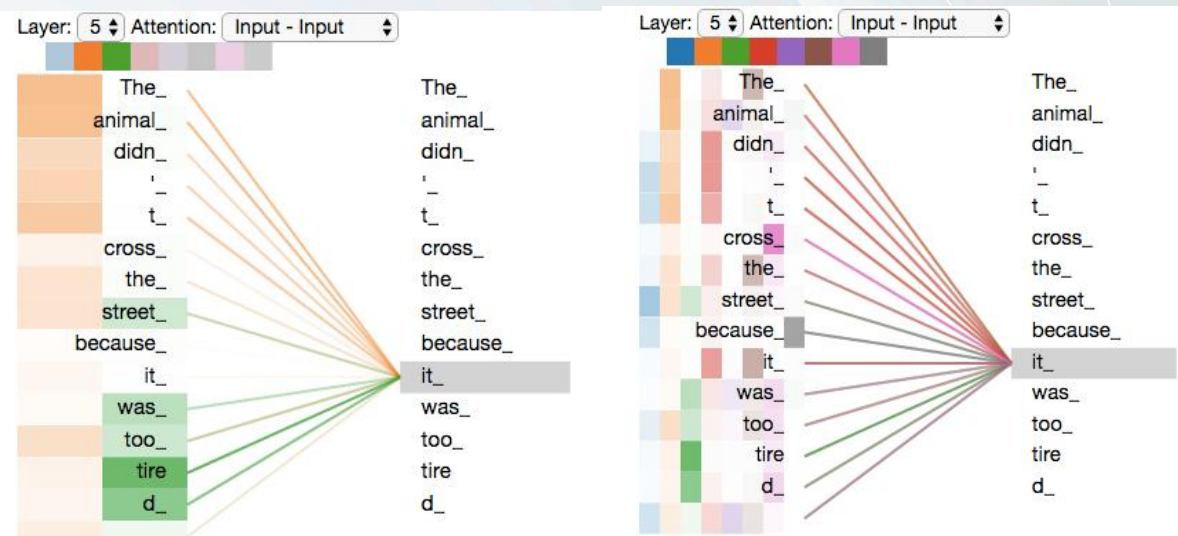
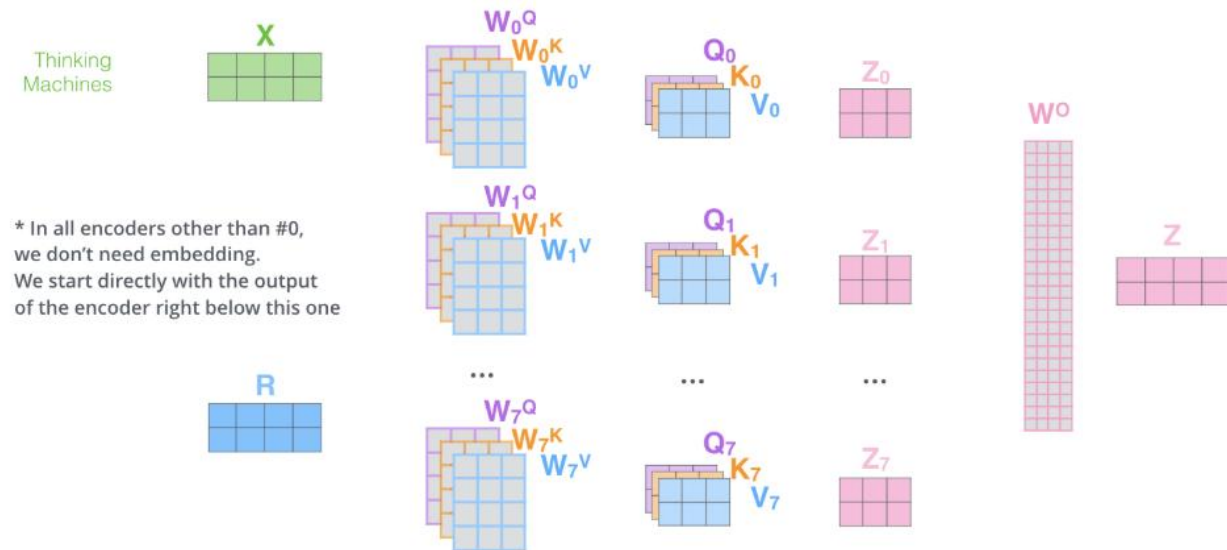
Transformer: Multi-Headed Attention (2/2)

Overview of Multi-Headed Attention

Example

- The animal didn't cross the street because it was too tired

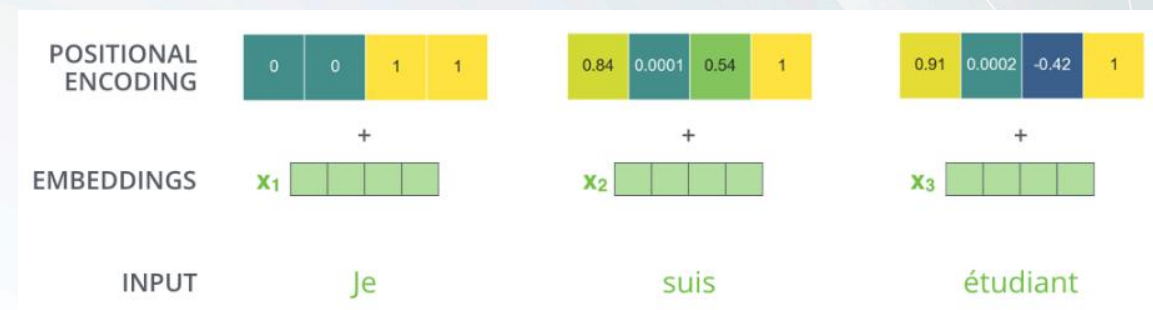
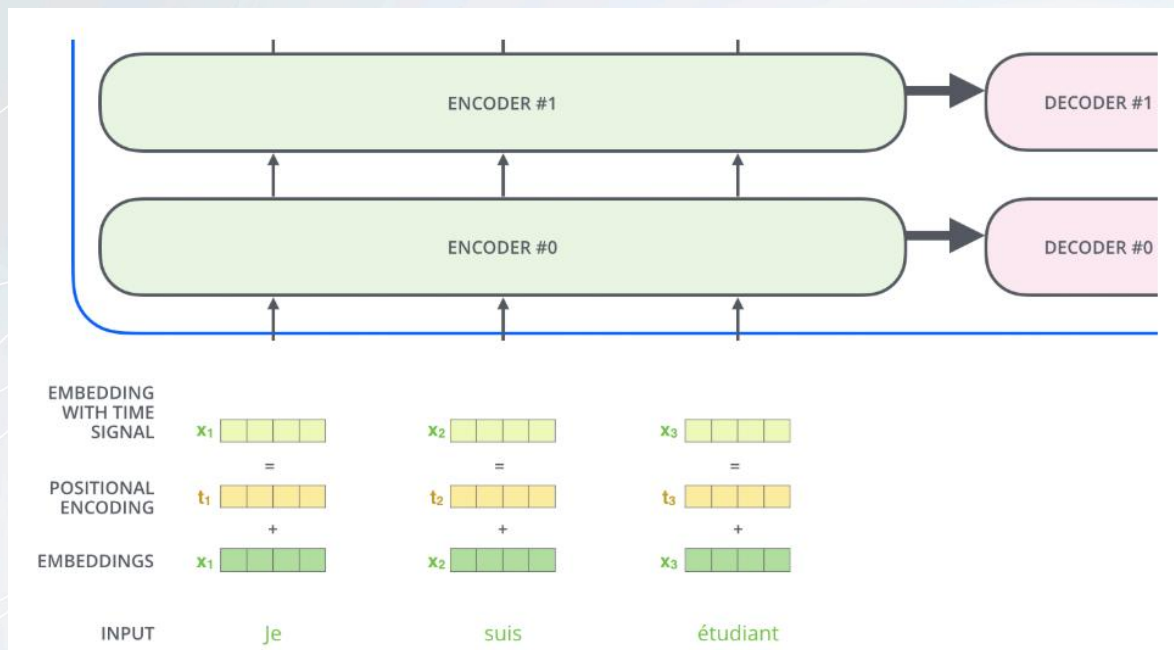
- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



Transformer: Positional Encoding (1/2)

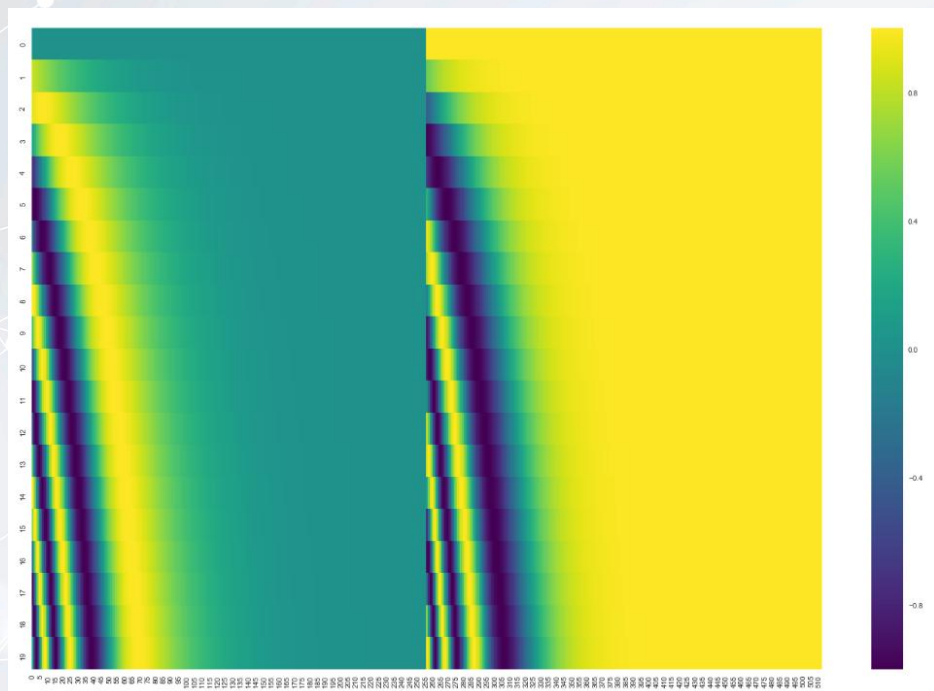
- Positional encoding vectors
 - the values of which follow a specific pattern

- Example
 - Dimensionality of 4

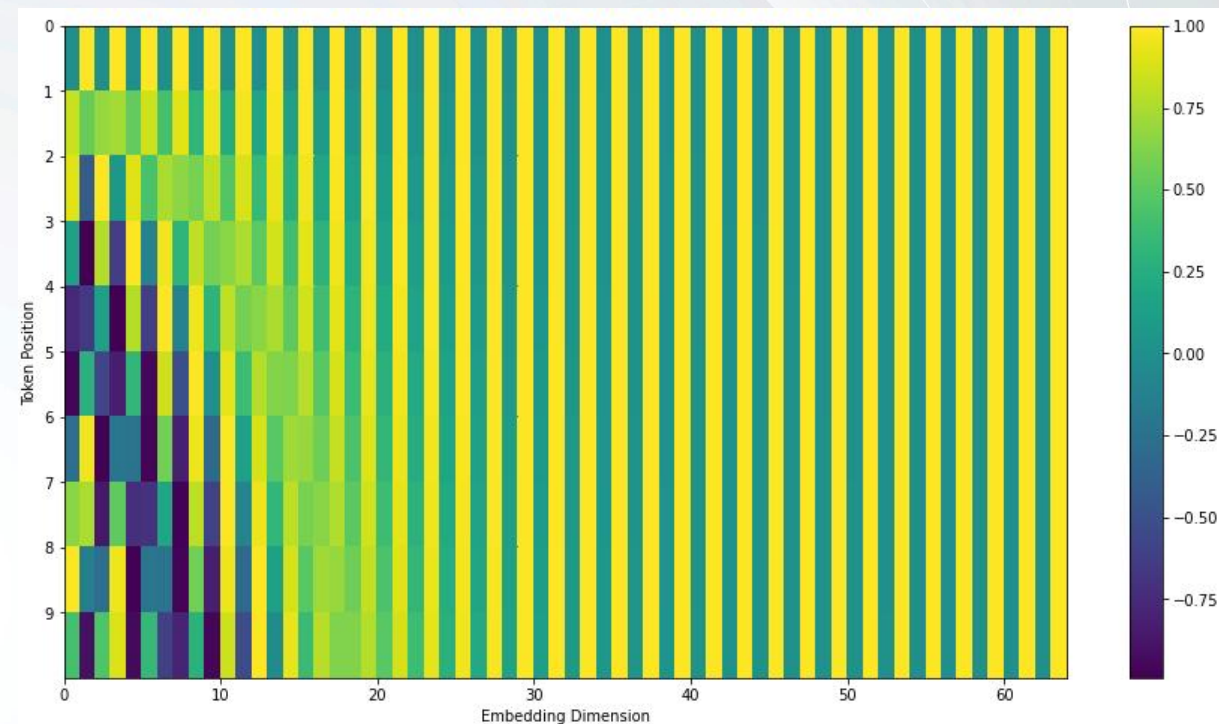


Transformer: Positional Encoding (2/2)

- Tensor2Tensor implementation
 - A real example of positional encoding for 20 words (rows) with an embedding size of 512 (columns)

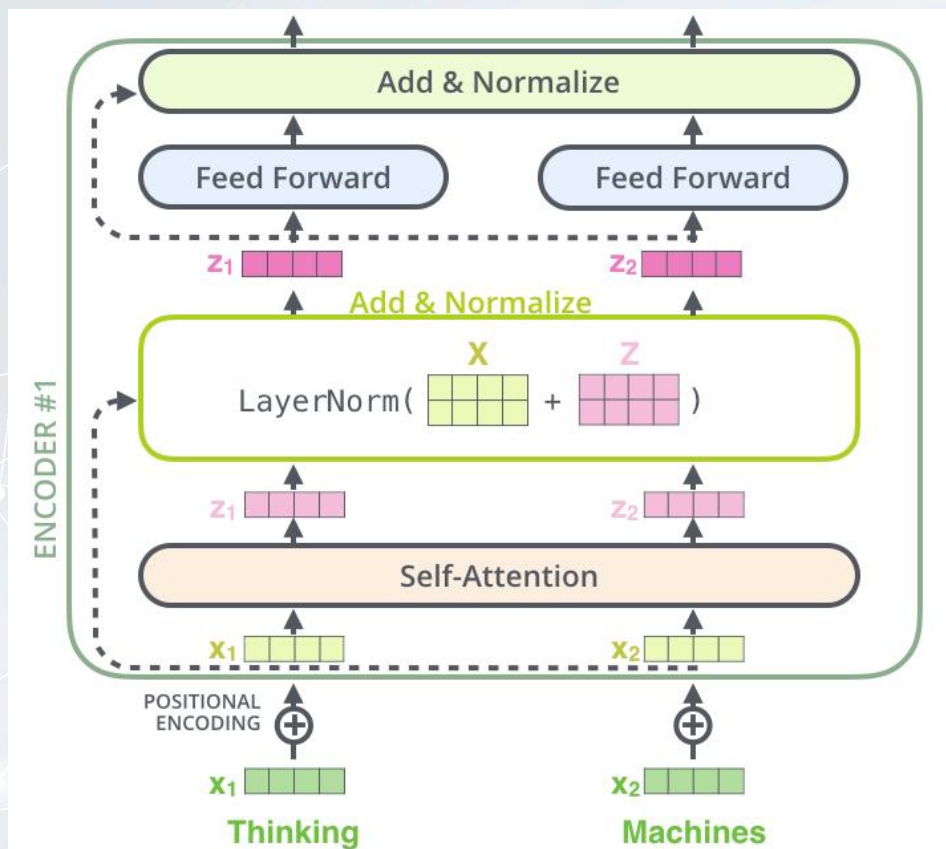


$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

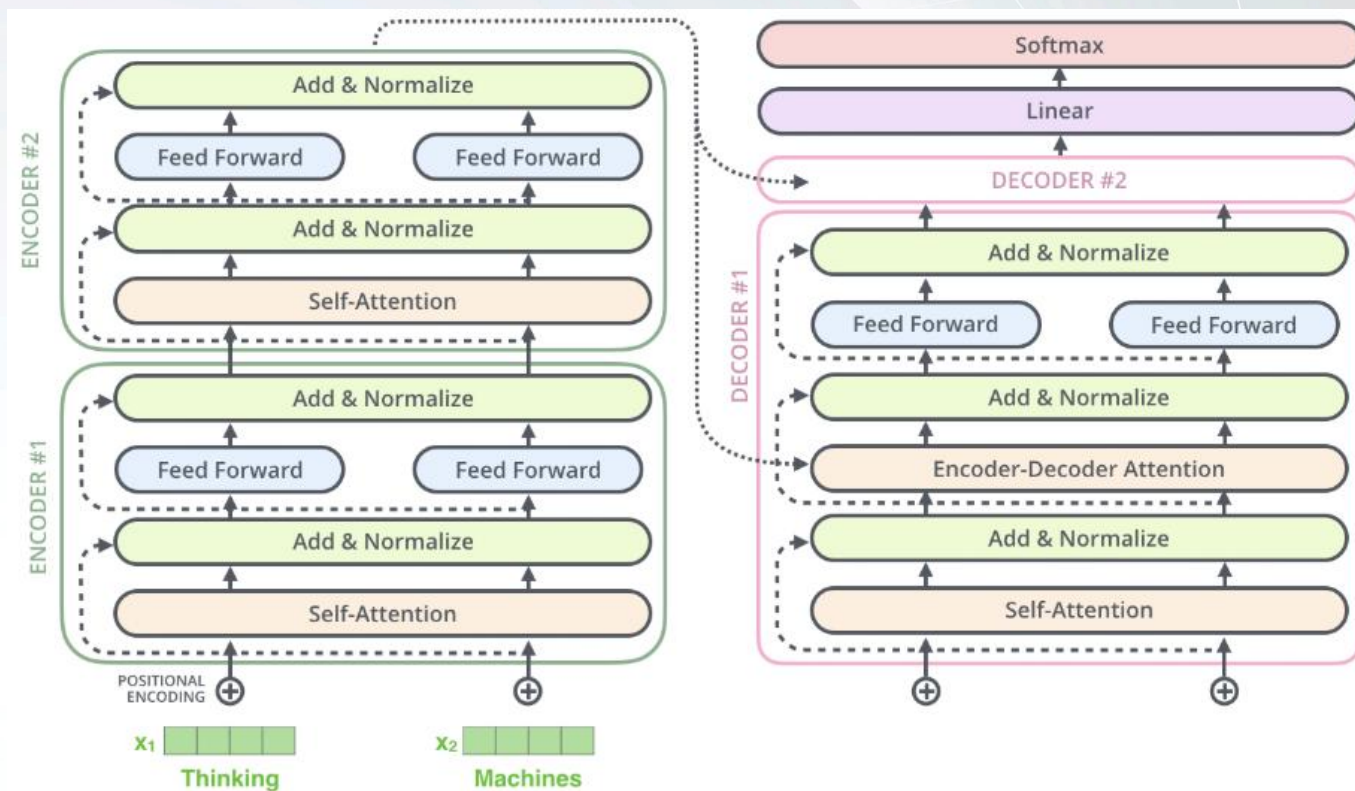


Transformer: The Residuals

- Residual Connections and Layer Normalization



- Example of a Transformer of 2 stacked encoders and decoders

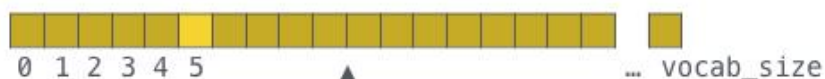


Transformer: The Final Linear and Softmax Layer

Which word in our vocabulary
is associated with this index?

Get the index of the cell
with the highest value
(argmax)

log_probs

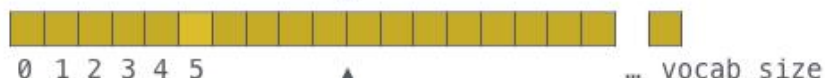


am

5

Softmax

logits



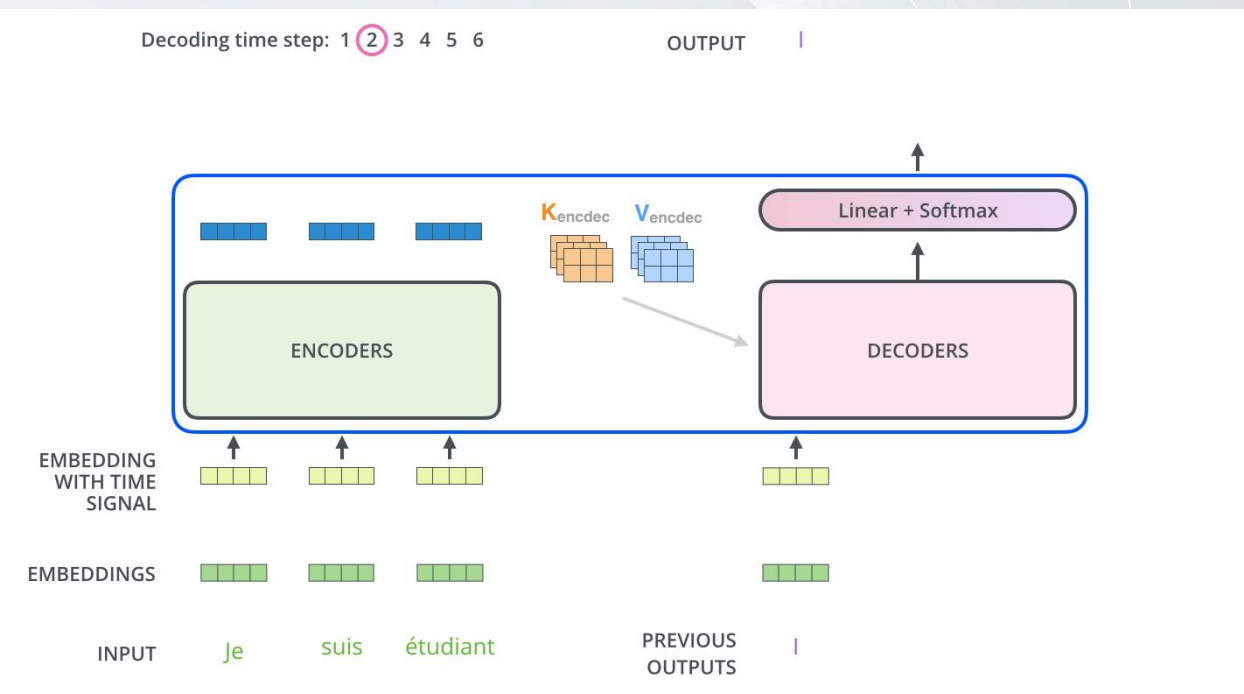
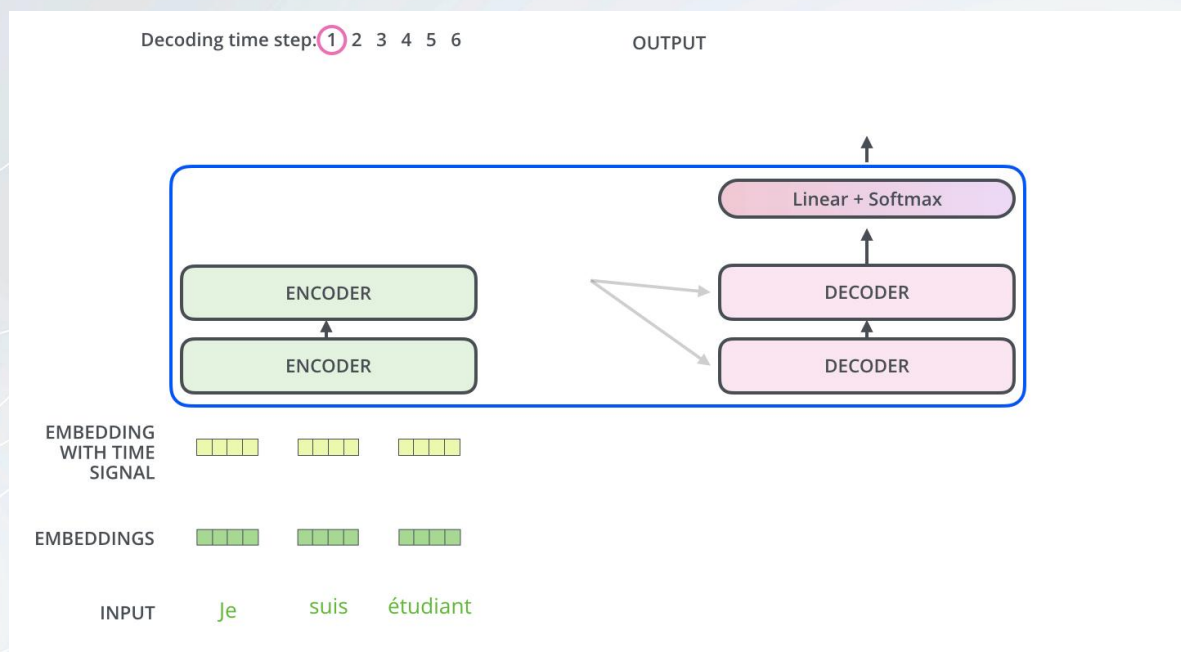
Linear

Decoder stack output



Generation Repeats One Simple Loop

- 1. Read the current token prefix
- 2. Predict a distribution for the next token
- 3. Select one token
- 4. Append it and run again



Transformer Components as Operators

Matrix multiplication

QKV projections
Attention score/value
products
FFN layers

Reductions

Softmax
LayerNorm statistics

Elementwise

Activation
Bias
Residual add

Data movement

Transpose / reshape
Masking
Reads and writes

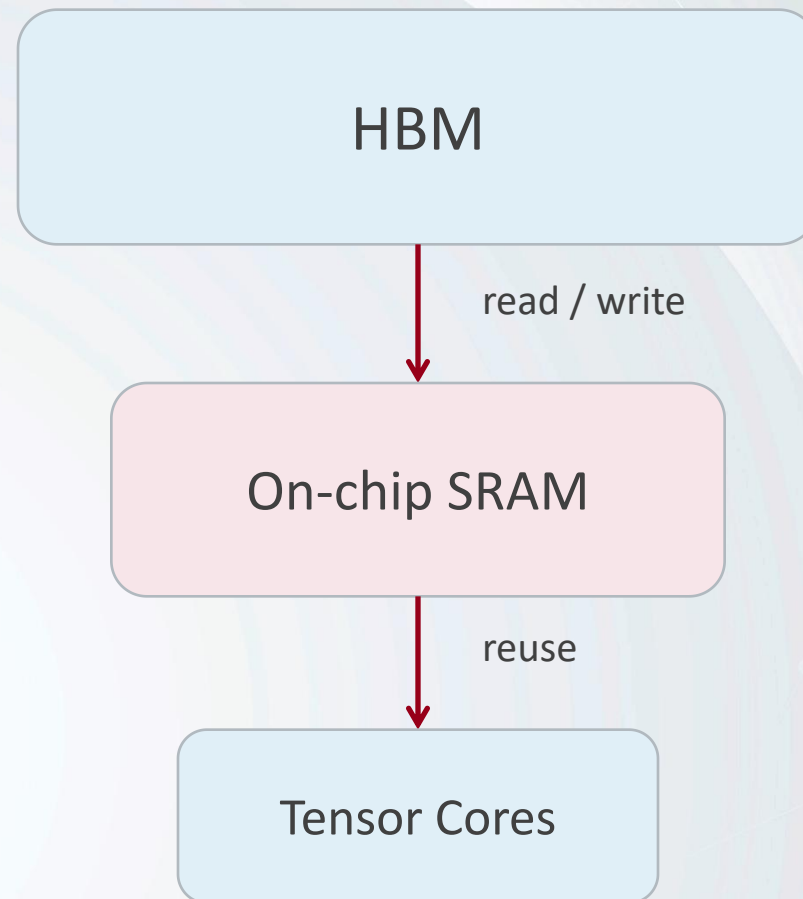
Performance depends on how these operators use compute and memory.

Data Movement Often Limits Attention

Q, K, and V are read from GPU memory.

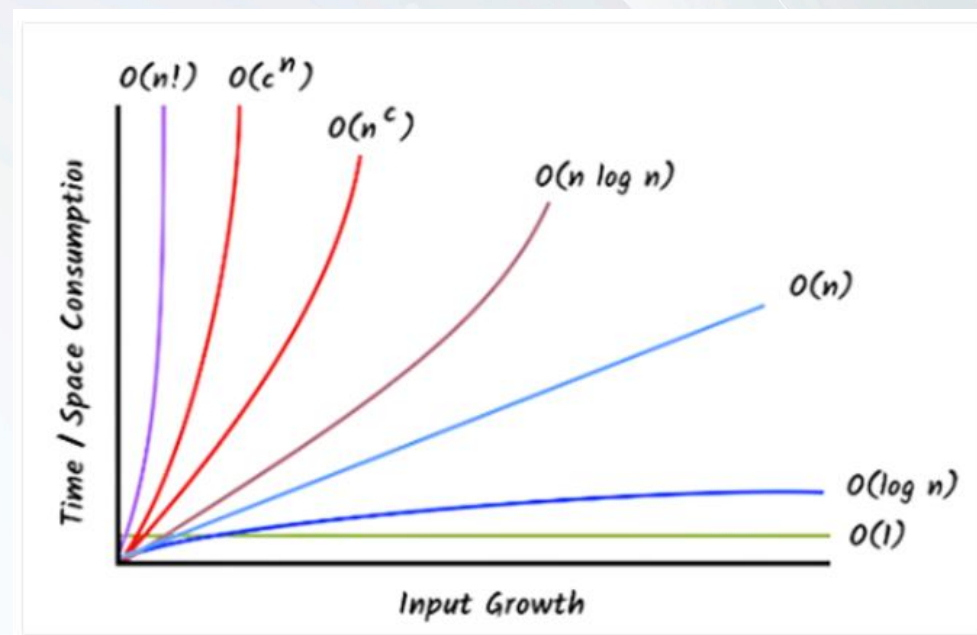
Large intermediates are written out and read back.

Moving bytes can dominate runtime.



• Complexity of Attention (1/2)

- In transformer self-attention, we compute attention scores between all pairs of tokens:
 - Query Q is an (n,d) matrix (for n tokens, each with dimension d).
 - Key K is an (n,d) matrix.
 - We form an $n \times n$ attention score matrix QK^T .
 - We then multiply by Value V , which is (n,d) matrix.
- This requires on the order of $n^2 d$ operations
- As n grows, the quadratic term dominates.



• Complexity of Attention (2/2)

• 1. Compute Attention Scores

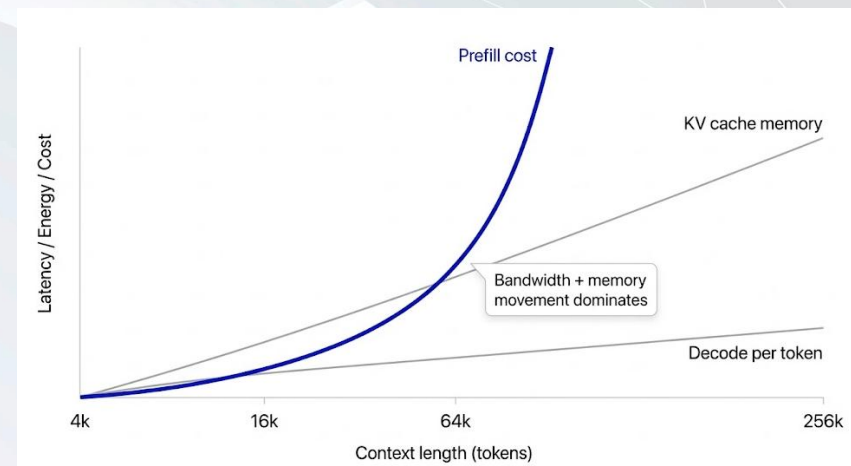
- $QK^T \Rightarrow (n,d) \times (d,n) = (n,n)$
- The cost: about $O(n \cdot d \cdot n) = O(n^2 d)$
- This yields a **score matrix** (n,n) (basically all pairs of tokens).

• 2. Multiply by Values, V

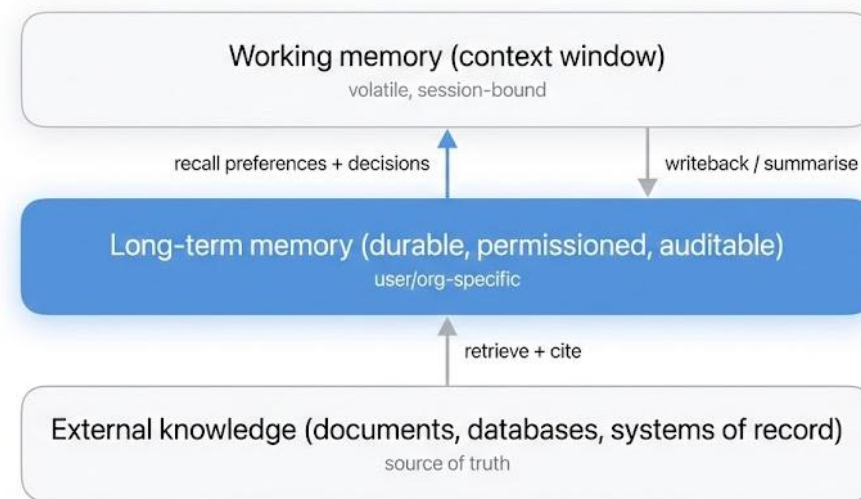
- **score matrix** $(n,n) \times V (n, d) = (n, d)$
- The cost again: $O(n \cdot n \cdot d) = O(n^2 d)$
- Hence **each forward pass** of a single-head attention block is about $O(n^2 d)$.
- If it's **multi-head** with **h heads**, each head uses dimension **d/h** but we do it **h** times. Summed together, it's again $O(n^2 d)$.

• The Memory Wall Problem

- Attention is expensive
 - Self-attention's flexibility comes at a steep cost: its complexity grows quadratically with the length of the input sequence.
- Where transformers hit the wall
 - KV Cache Explosion
 - The Bandwidth Bottleneck
- Attention is all you need until you need memory

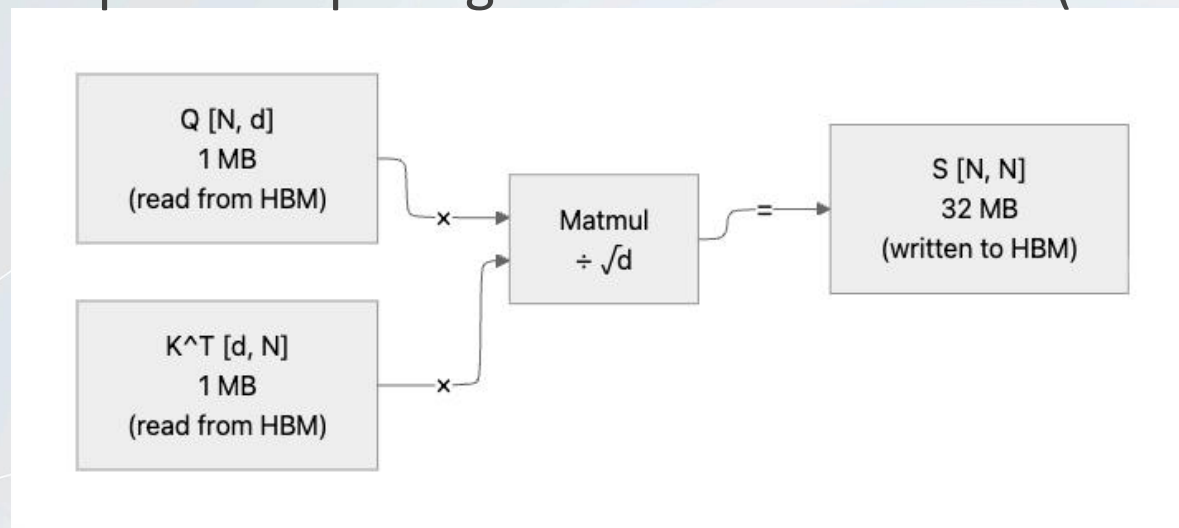


“Memory Isn’t One Thing”



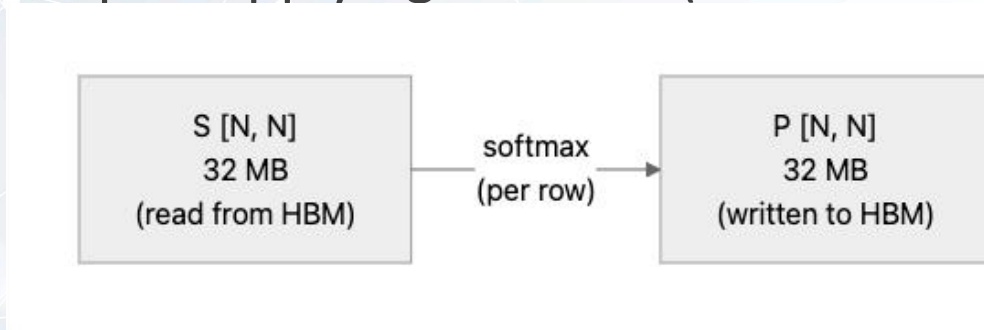
• Data Flow Diagram of Standard Attention (1/2)

- Step 1: Computing the Attention Scores ($S = QK^T / \sqrt{d}$)



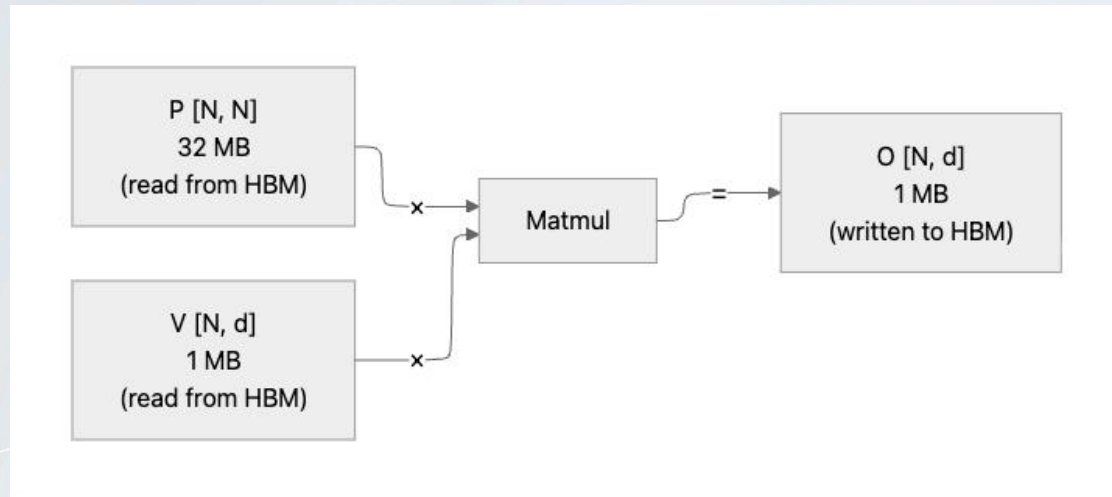
For example:
 $N = 4096, d = 128, \text{FP16}$

- Step 2: Applying Softmax ($P = \text{softmax}(S)$)



• Data Flow Diagram of Standard Attention (2/2)

- Step 3: Computing the Output ($O = PV$)



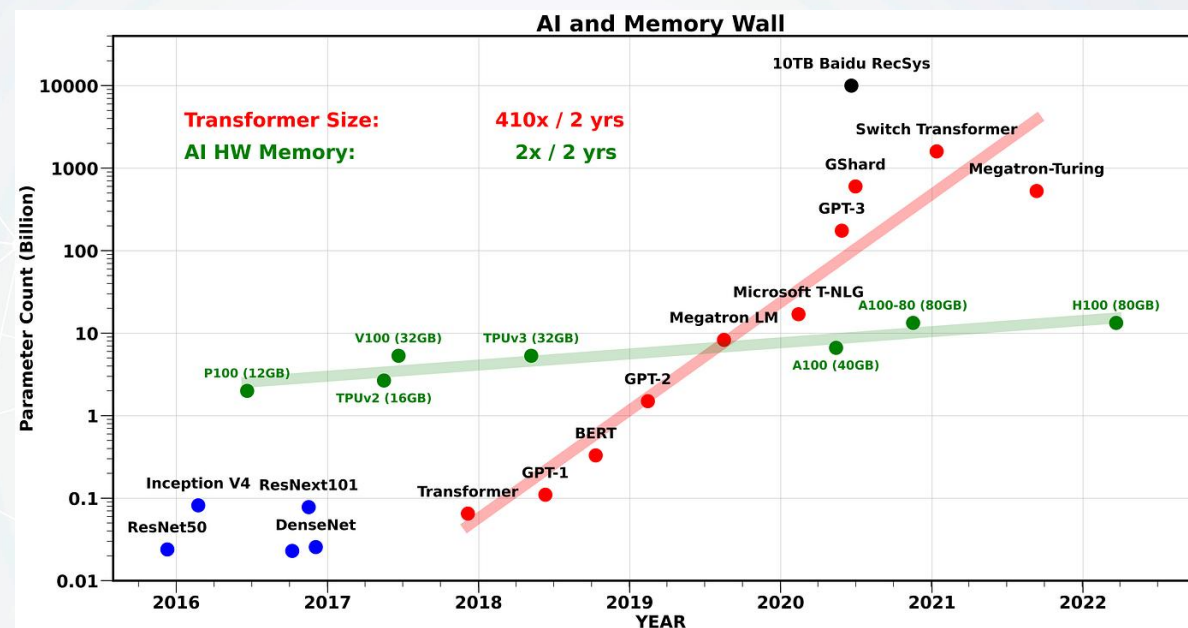
For example:
 $N = 4096$, $d = 128$, FP16

- Total HBM Traffic:

Step	Reads from HBM	Writes to HBM	Subtotal
Step 1: $S = QK^T / \sqrt{d}$	Q (1 MB) + K (1 MB)	S (32 MB) $\leftarrow N \times N$ intermediate	34 MB
Step 2: $P = \text{softmax}(S)$	S (32 MB) $\leftarrow N \times N$ intermediate	P (32 MB) $\leftarrow N \times N$ intermediate	64 MB
Step 3: $O = PV$	P (32 MB) + V (1 MB)	O (1 MB)	34 MB
Total			132 MB

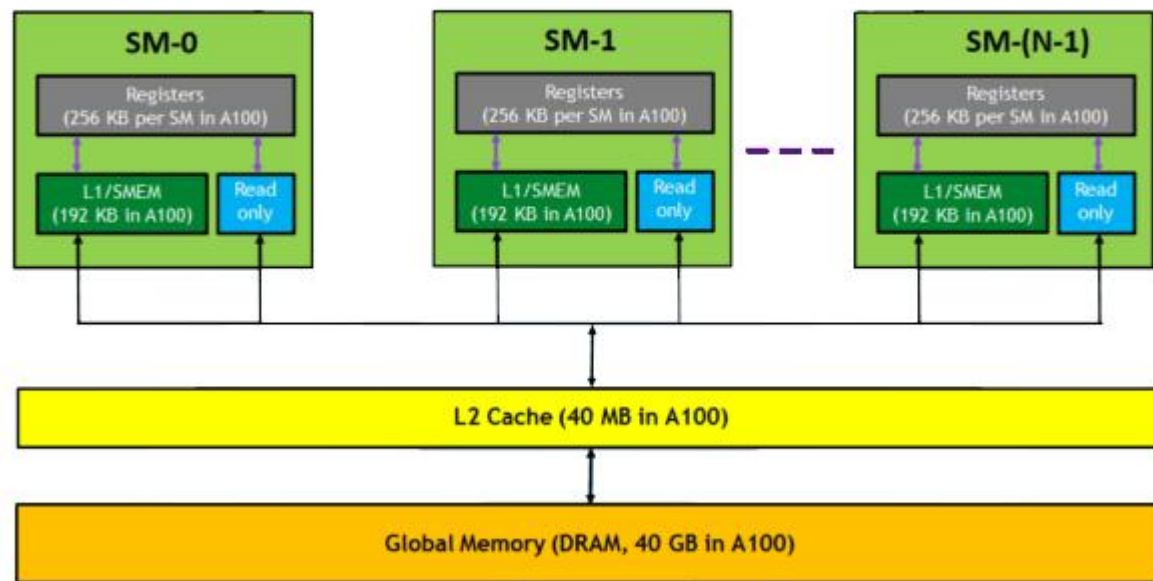
• The "Memory Wall" of Standard Attention

- The Problem: Standard Attention complexity is $O(n^2)$.
- Pain Points: Memory consumption scales quadratically with sequence length n , leading to frequent Out-of-Memory errors during long-context processing.
- The Crucial Question: Do we truly need to materialize and store that massive $N \times N$ attention matrix?



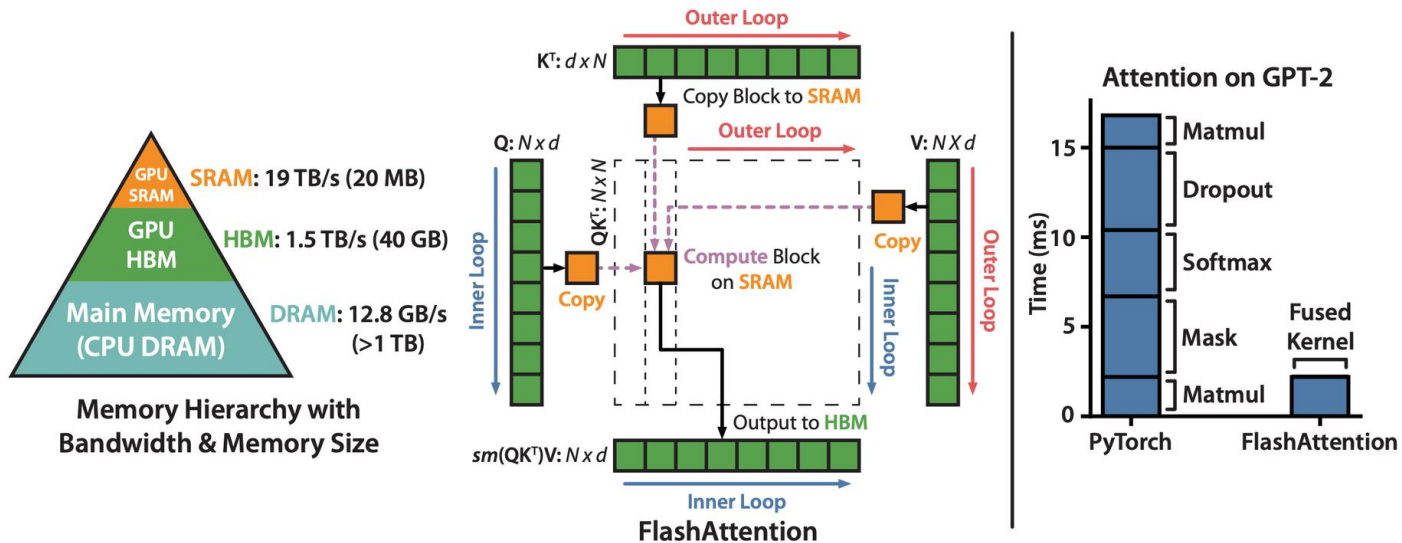
• GPU Memory Hierarchy Explained

- Speed Disparity: HBM vs. SRAM (10x Factor)
 - For example: A100
 - GPU Memory / HBM2e: 40/80 GB 1.5-2 TB/s
 - L2 Cache: 40 MB ~5 TB/s
 - L1 Cache / Shared Memory: 192 KB/SM 19 TB/s



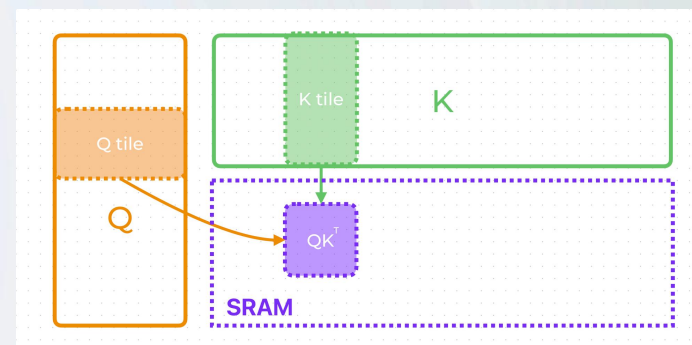
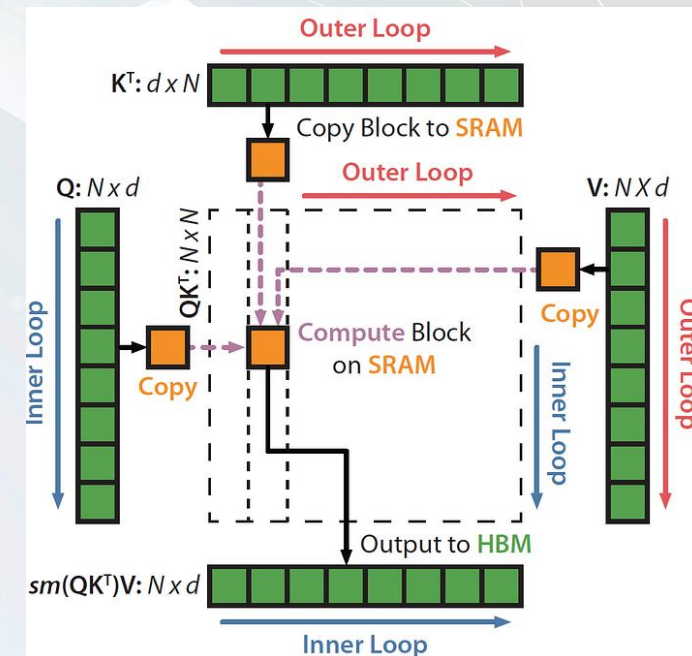
Core Idea of Flash Attention: Less movement

- Less Movement, More Speed
 - Design Philosophy: Optimize Data Movement, Not Just Computation
- Two Technical Pillars:
 - Tiling: Breaking down large tasks into smaller blocks that fit the SRAM capacity.
 - Recomputation: Instead of storing intermediate activation values during the forward pass, they are recalculated during the backward pass to save memory space.



• Tiling Concept

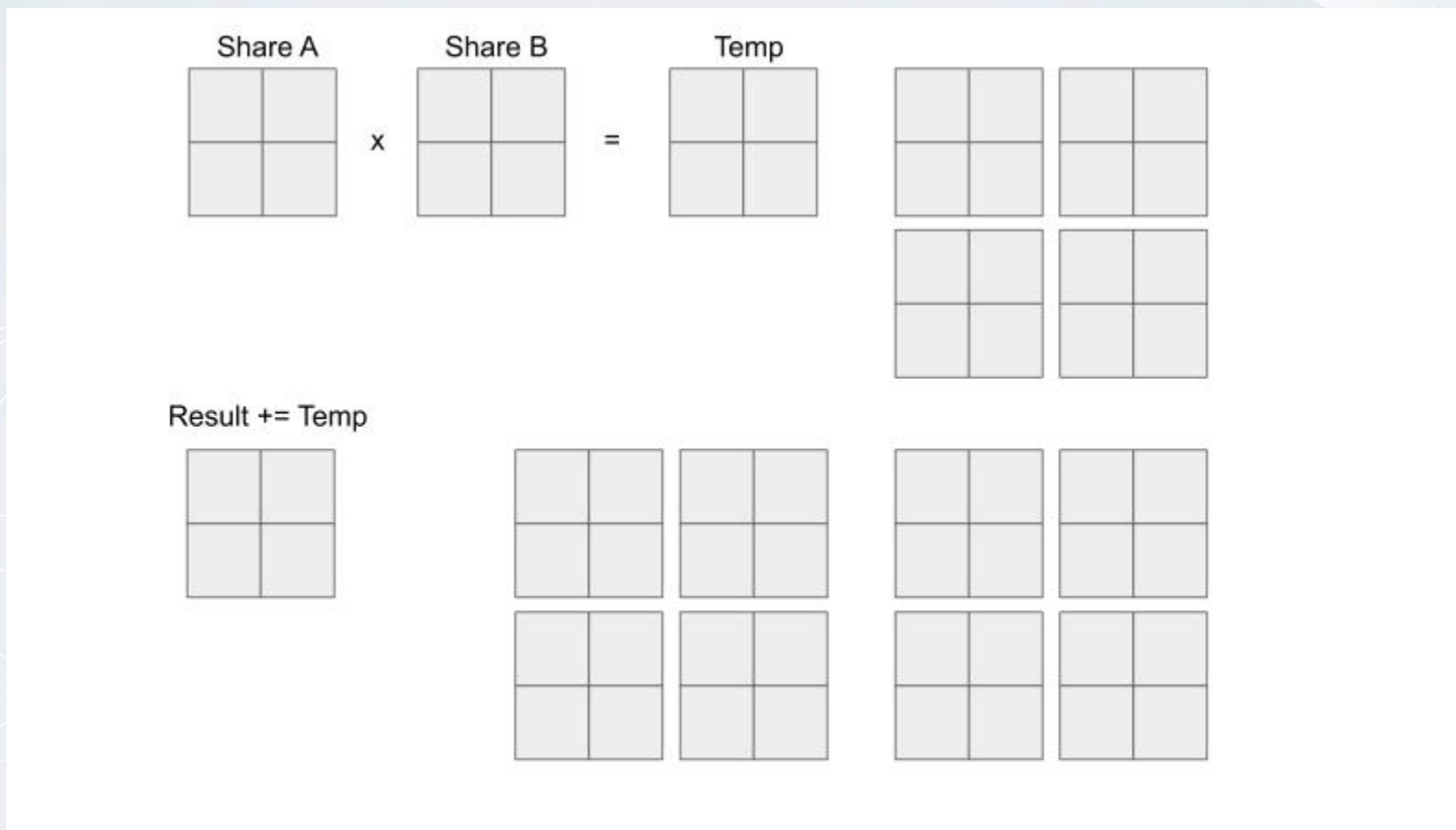
- Why Tiling?
 - GPU SRAM (e.g., L1/Shared Memory in A100) is extremely fast but has very limited capacity (typically a few hundred KB).
 - Traditional $N \times N$ matrices (e.g., when $N=4096$) far exceed SRAM capacity and must be partitioned into "tiles" that can fit into the cache.
- How it Works?
 - Input matrices Q , K , and V are partitioned into blocks (tiles). Using nested loops, these blocks are loaded from HBM to SRAM piece by piece for local computation.
 - Performance Gain: Partial dot-products and Softmax accumulations are completed entirely within SRAM. This eliminates the need to write $N \times N$ intermediate results back to the slow HBM.



• Dynamic Demo



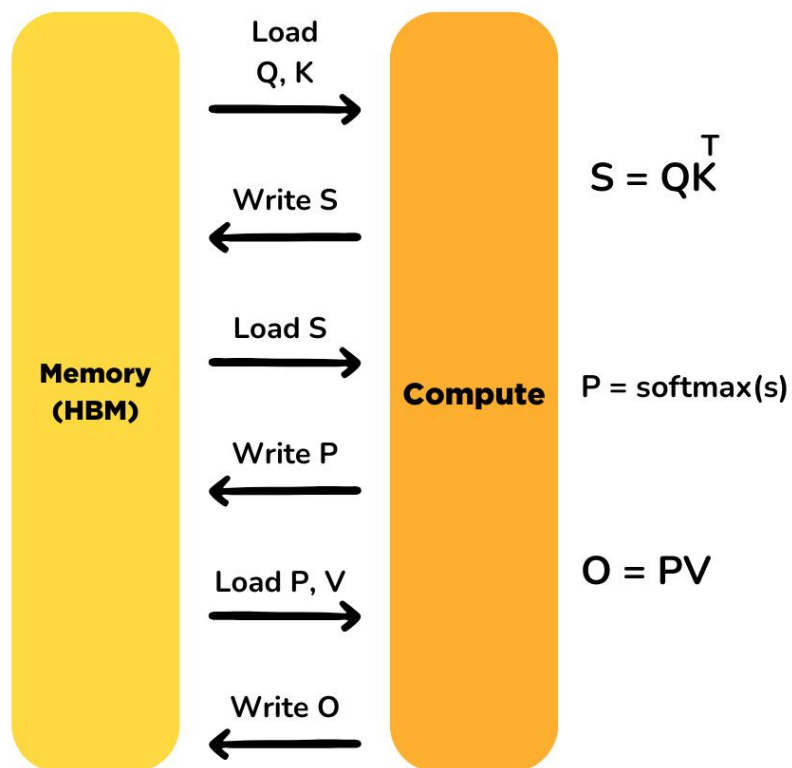
- The process of loading tiled blocks.



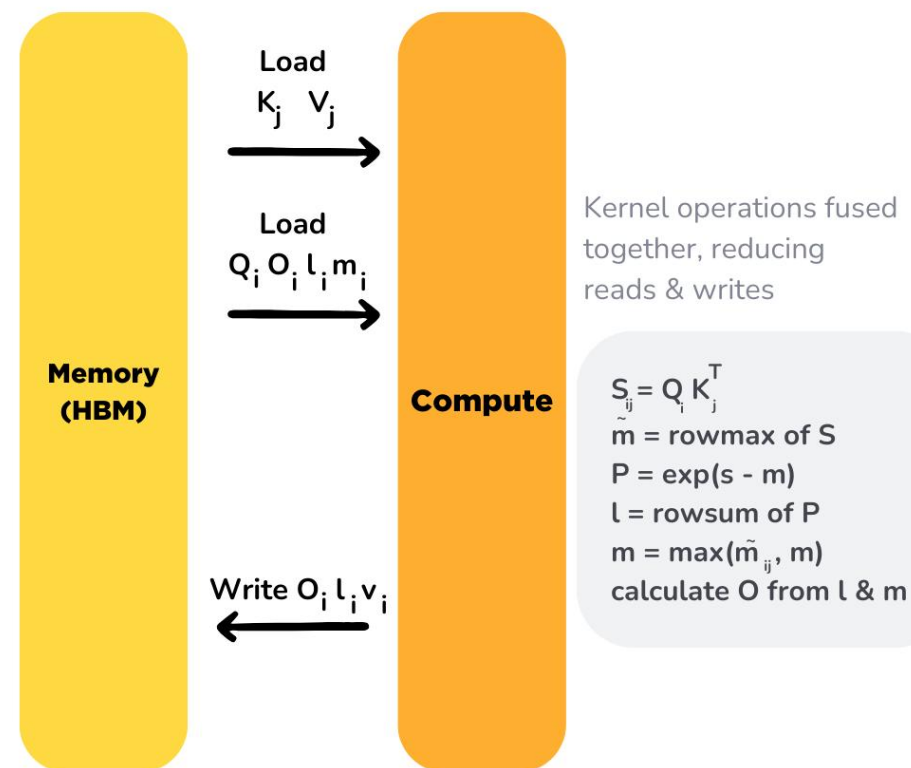
Memory Access Comparison

Standard Attention vs. Flash Attention

Standard Attention Implementation



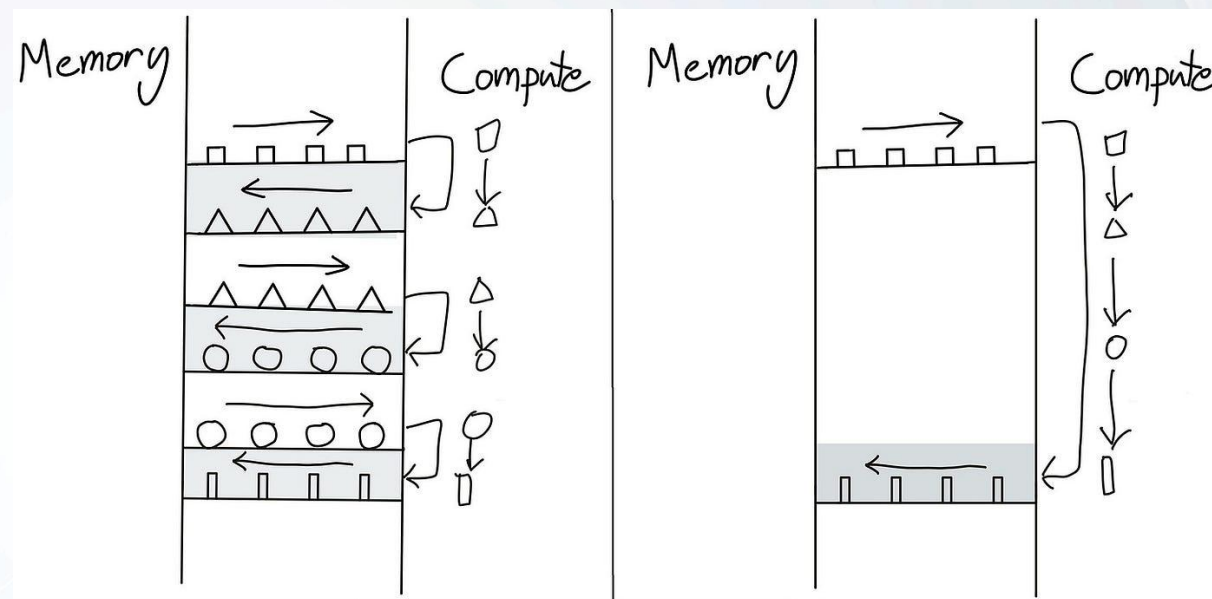
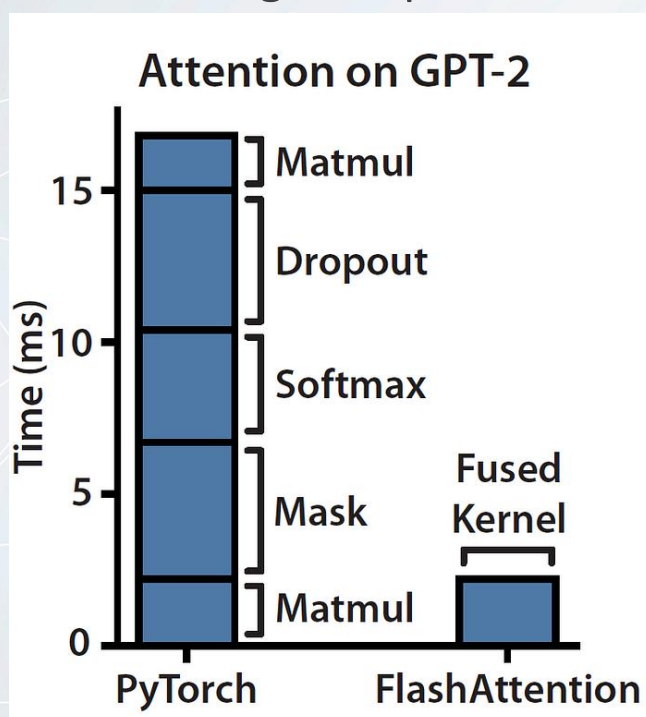
Flash Attention



Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

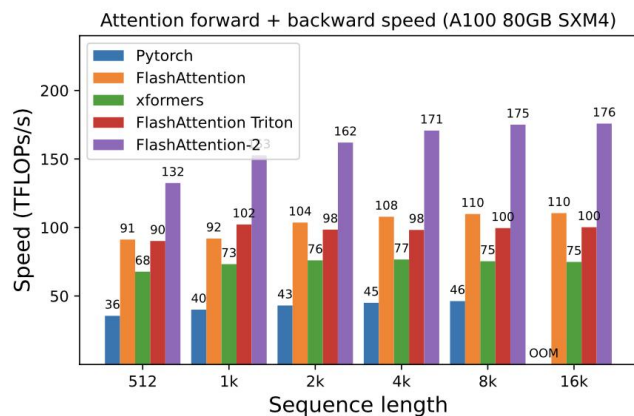
• Kernel Fusion

- Standard attention launch separate kernels
 - Each launch clears the SRAM and writes results back to HBM, causing massive I/O waste.
- Flash attention fuse all logic into one highly optimized kernel
 - SRAM-Centric Execution: Perform tiled loading, dot-product, online softmax, and weighted sum within a single loop.

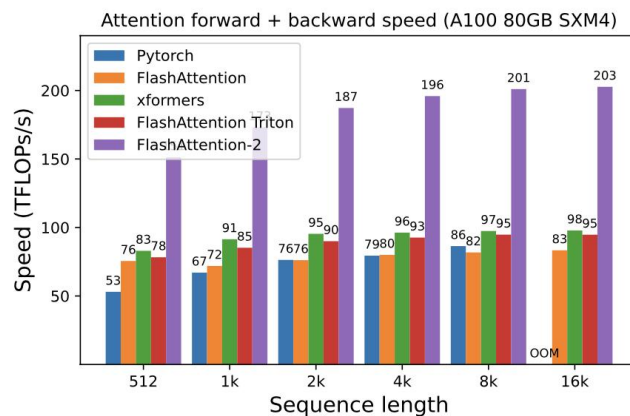


Operator Fusion Simplified

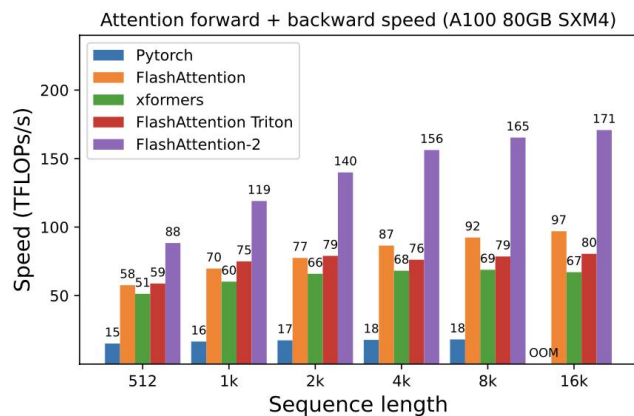
Performance Benchmarks & Throughput



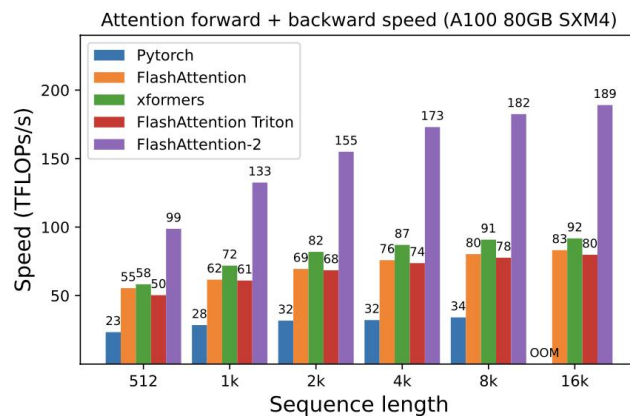
(a) Without causal mask, head dimension 64



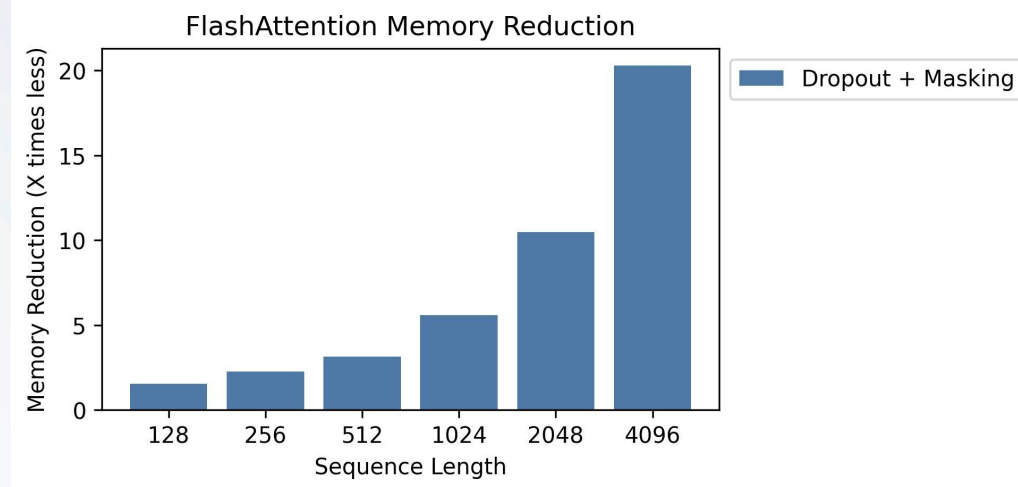
(b) Without causal mask, head dimension 128



(c) With causal mask, head dimension 64



(d) With causal mask, head dimension 128



Operator Fusion

- Operator Fusion
- Example: RMSNorm + Residual Fused Operator



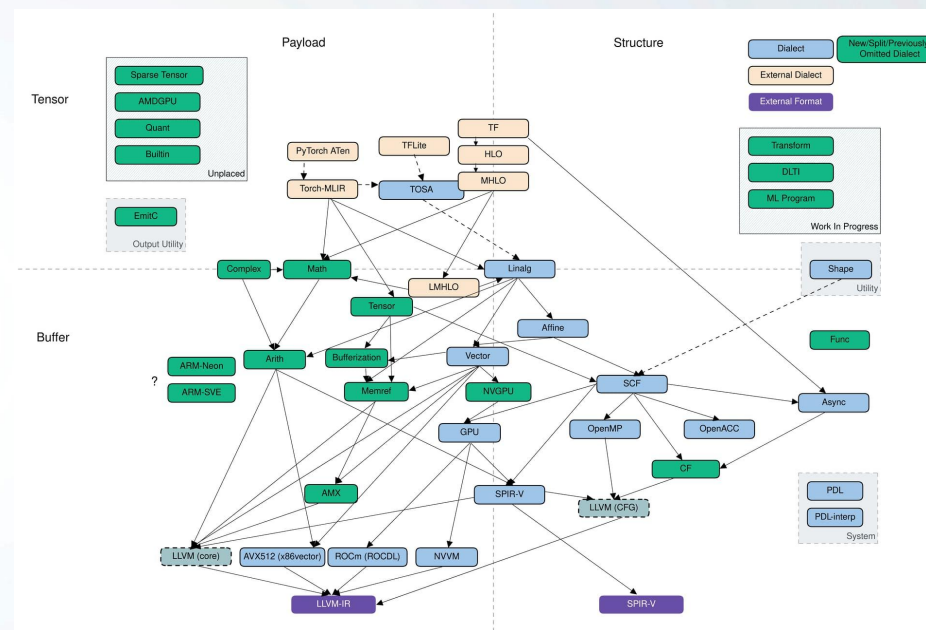
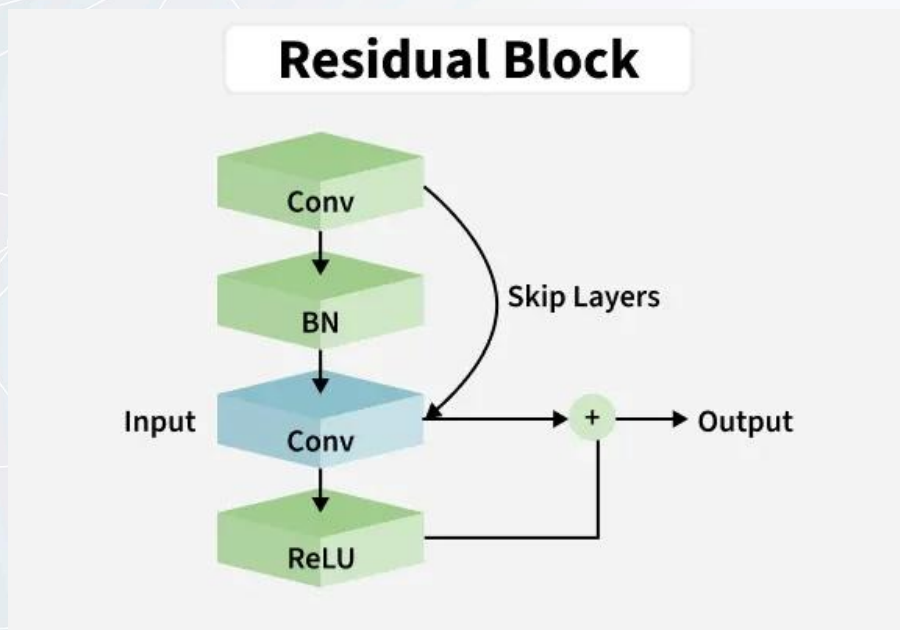
BAAI
智源研究院



北京大学
PEKING UNIVERSITY

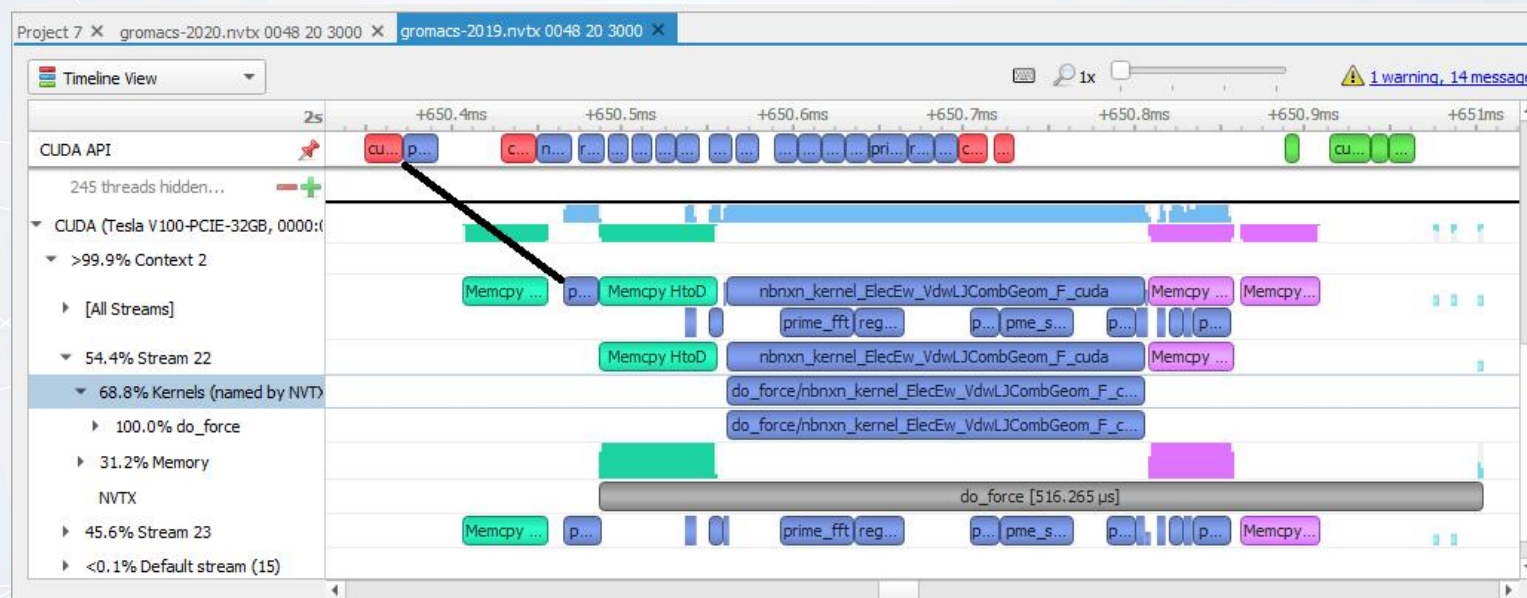
• Lifecycle of Operators

- Frame layer:
 - Data graph normalization: Break down high-level composite operators in deep neural networks (e.g., TransformerBlock) into fine-grained primitives (MatMul, Softmax, LayerNorm), and establish standardized IR frameworks (e.g., MLIR, XLA HLO).
 - Calculate the tensor lifecycle through data stream analysis and determine the parallel strategy
 - Convert the optimized compute graph Lowering into device-specific code and insert necessary synchronization primitives



• Lifecycle of Operators

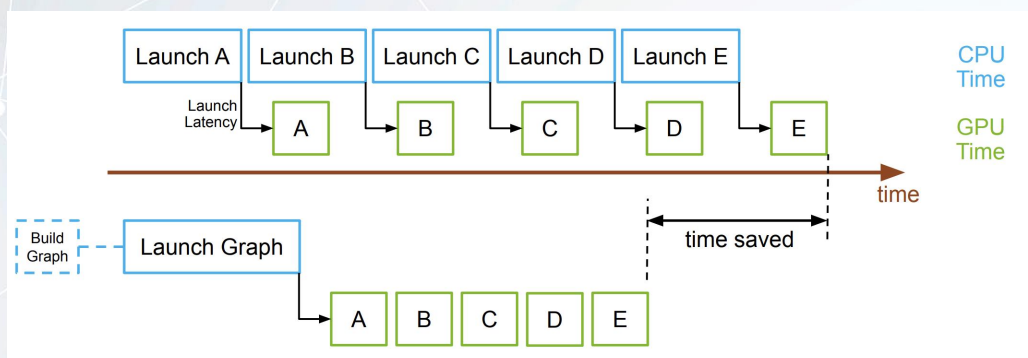
- Runtime layer:
 - Prepare parameters for each operator (device pointer, shared memory size, Stream ID, and other context)
 - Initiate Kernel Launch
- hardware layer :
 - GPU reads data from Global Memory---> compute unit processes---> writes back to Global Memory



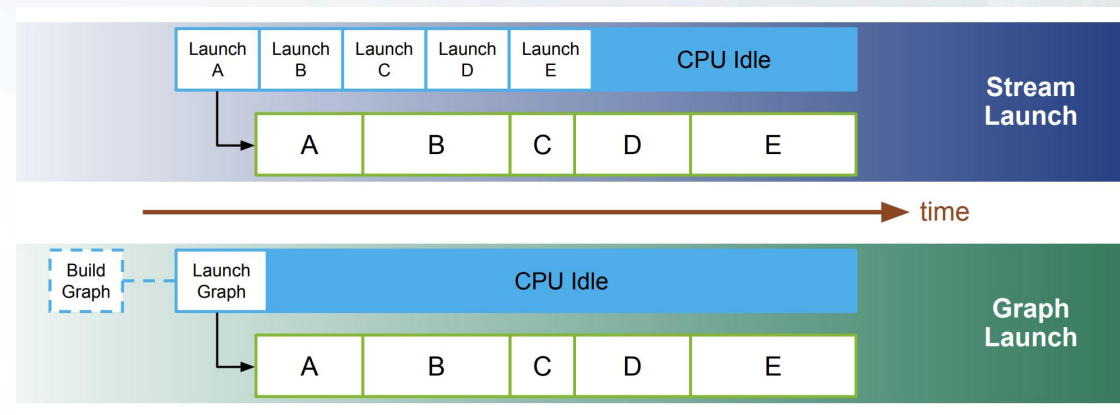
Kernel Launch Optimization: CUDA Graph

- Record a series of GPU operations (Kernels, Memcpys) as a static topology diagram
 - Capture the execution sequence, parameters, video memory addresses, and scheduling paths of all kernels during the phase.
 - During the replay phase, the CPU only needs to issue a single ultra-lightweight instruction to trigger the entire graphs automatic transfer on the GPU.

minimal GPU kernel

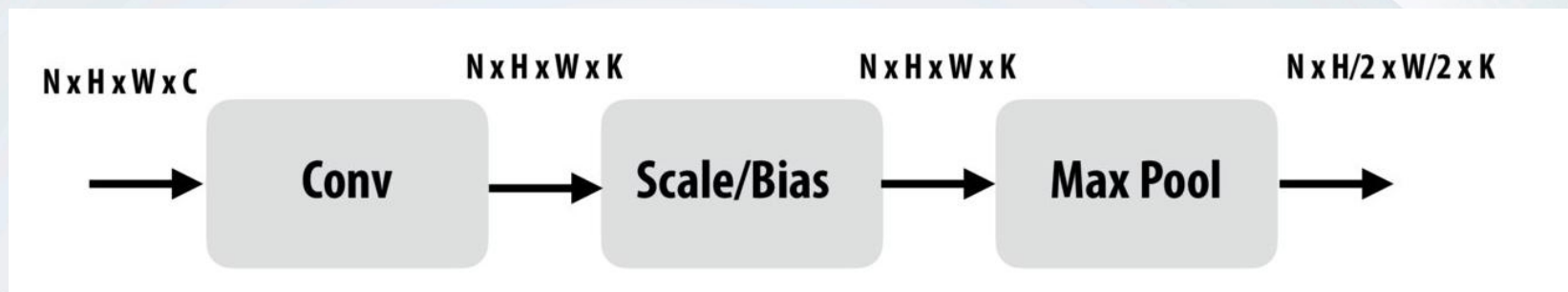


general-purpose GPU kernel



• Data transfer between operators

- Default mode: Conv results must be written back to global video memory, then Scale reads them out



- Operator fusion: Element-wise operations can seamlessly integrate with other operators, significantly reducing both memory access and kernel startup overhead.



• Operator Fusion



- Definition: A technique that merges multiple independent operators (or kernels) in a computational graph into a single operator for execution.
- Typical scenarios in deep learning:
 - Vertical Fusion: Conv + Bias + ReLU
 - Horizontal Fusion: Merge multiple GEMMs with the same input but different parameters
 - Reduction Fusion: Mean + Var + Scale in LayerNorm

• Advantages of Operator Fusion

- Reduce memory bandwidth usage: Eliminate read and write operations on global memory for intermediate results. Data remains in registers or shared memory.
- Reduce Kernel Launch overhead: Minimize the number of handshakes between CPU and GPU
- Enhance cache utilization: Data is continuously accessed in SRAM for higher efficiency
- Increase computational density: Boost the instruction stream per task to better mask memory latency

• Problems of Operator Fusion

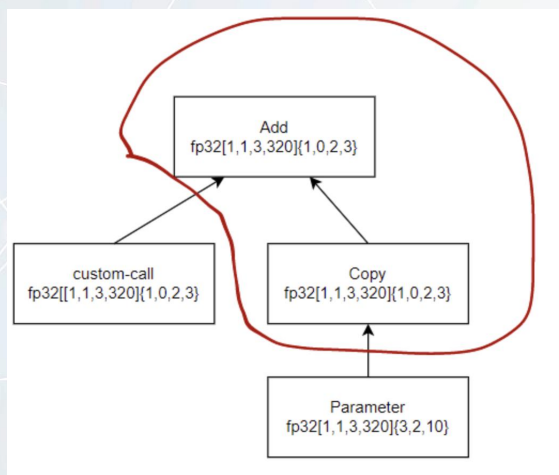


- Core contradiction-Register pressure
 - The total number of registers in a GPU SM is fixed (for example, A100 has 256KB per SM).
 - Single-threading requires more intermediate states → Single-threading registers ↑ → Each thread block occupies registers ↑ → Concurrent thread blocks decrease → Reduced active Warp count in SM (Occupancy decreases)
- Instruction cache pressure:
 - The merged kernel may exceed the hardware instruction cache limit.
- Finding the sweet spot between saving memory bandwidth and maintaining sufficient computational parallelism

• A Negative Example: The Greedy Merging of XLA



- Scene description
 - The inputs forced layout is {3,2,1,0}, and the operator (Custom-call) has a forced layout of {1,0,2,3}.
 - When computing Add, XLA detects that the two inputs have different layouts. To align them, it must insert a Copy operation.
- To reduce kernel boot overhead, XLA forcibly merges Add and Copy operations. However, inconsistent I/O layouts disrupt memory merge access, causing memory access to become random and erratic, resulting in a 9-fold performance degradation.



Fused computation	I/O tensors with different layouts	I/O tensors use the same layout {1,0,2,3}	I/O tensors use the same layout {3,2,1,0}
fused_computation.1116	21.63ms	2.43ms	2.43ms

• Example: RMSNorm + Residual

```
def naive_add_rms_norm(x, residual, weight, eps=1e-6):
    """
    Compute fused Residual + RMSNorm using native PyTorch.

    This follows exactly the computation order of the Triton kernel.
    """
```

```
norm_fusion_lab MN elements ; write MN elements
updated = x + residual

# save updated residual
updated_residual = updated

# read MN elements ; write M elements
variance = (updated * updated).mean(dim=1)

# read M elements ; write M elements
rrms = torch.rsqrt(variance + eps)

# read MN + M + N elements ; write MN elements
out = updated * rrms[:, None] * weight

# in total:
# read 5MN + 2M + N elements
# wrote 2MN + 2M elements
return out, updated_residual
```

• Formula

RMSNorm

$$\text{RMSNorm}(\mathbf{x}) = \mathbf{y} = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x}) + \epsilon} \odot \mathbf{w}$$

$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}$$

- Performance bottleneck: Total reads $5MN + 2M + N$ elements, writes $2MN + 2M$ elements. Frequent global memory reads/writes cause significant bandwidth waste.

• Example: RMSNorm + Residual



- Basic Idea of Operator Fusion
 - Load: All threads collaborate in parallel to read data from Global Memory and write it to Shared Memory
 - Sync & Reduce: Perform multiple iterations within Shared Memory, which is fully executed at high speed on-chip without consuming video memory bandwidth.
 - CUDA: Synchronize with `__syncthreads()` at each step
 - Triton: Automatic
 - Write: Only the final result is written back to Global Memory.

• fused_add_rms_norm_kernel

- SRAM Residency: The entire row of data remains on-chip after loading until the computation is completed.

```
@triton.jit
def _fused_add_rms_norm_kernel(
    x_ptr,
    residual_ptr,
    out_ptr,
    updated_residual_ptr,
    weight_ptr,
    n_cols: tl.constexpr,
    eps,
    BLOCK_SIZE: tl.constexpr,
```

```
):
    pid = tl.program_id(0)
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < n_cols
    row_offsets = pid * n_cols + cols
```

memory
pointer
calculation

```
x = tl.load(x_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
residual = tl.load(residual_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
weight = tl.load(weight_ptr + cols, mask=mask, other=0.0).to(tl.float32)
```

loading

```
updated = x + residual
tl.store(updated_residual_ptr + row_offsets, updated, mask=mask)
```

```
variance = tl.sum(updated * updated, axis=0) / n_cols
rrms = 1.0 / tl.sqrt(variance + eps)
out = updated * rrms * weight
tl.store(out_ptr + row_offsets, out, mask=mask)
```

Data writeback

• fused_add_rms_norm_kernel

- Operator Fusion: Merge Max, Sub, Exp, and Sum into a Kernel

```
@triton.jit
def _fused_add_rms_norm_kernel(
    x_ptr,
    residual_ptr,
    out_ptr,
    updated_residual_ptr,
    weight_ptr,
    n_cols: tl.constexpr,
    eps,
    BLOCK_SIZE: tl.constexpr,
):
    pid = tl.program_id(0)
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < n_cols
    row_offsets = pid * n_cols + cols

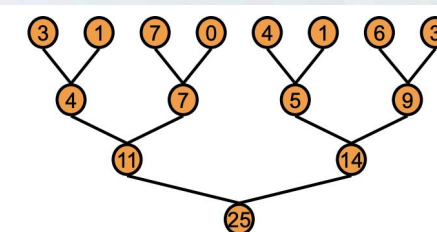
    x = tl.load(x_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
    residual = tl.load(residual_ptr + row_offsets, mask=mask, other=0.0).to(tl.float32)
    weight = tl.load(weight_ptr + cols, mask=mask, other=0.0).to(tl.float32)

    updated = x + residual
    tl.store(updated_residual_ptr + row_offsets, updated, mask=mask)

    variance = tl.sum(updated * updated, axis=0) / n_cols
    rrms = 1.0 / tl.sqrt(variance + eps)
    out = updated * rrms * weight
    tl.store(out_ptr + row_offsets, out, mask=mask)
```

operator
fusion

Internal Automatic
Realization of Parallel
Tree Structure



• RMSNorm + Residual Host

```
def _block_size(n_cols):  
    # 计算 Block 大小 (静态)  
    # 取能够覆盖一整行的最小 2 的幂  
    return triton.next_power_of_2(n_cols)  
  
def fused_add_rms_norm(x, residual, weight, eps=1e-6):  
    # 为输出结果提前分配显存  
    out = torch.empty_like(x)  
    updated_residual = torch.empty_like(x)  
  
    # 获取矩阵尺寸  
    n_rows, n_cols = x.shape  
  
    # 1. 确定 Block 大小 (静态)  
    # 一个 Triton Program 负责计算一整行数据,  
    # Block 大小取能够覆盖 n_cols 的最小 2 的幂。  
    block_size = _block_size(n_cols)  
  
    # 2. 启动 Kernel (动态)  
    # Grid 大小等于矩阵行数, 每个 Program 处理一行:  
    # (1) Residual Add  
    # (2) 计算 RMS  
    # (3) 完成归一化  
    # (4) 与可学习权重相乘  
    _fused_add_rms_norm_kernel[(n_rows,)](  
        x,  
        residual,  
        out,  
        updated_residual,  
        weight,  
        n_cols,  
        eps,  
        BLOCK_SIZE=block_size,  
    )  
  
    return out, updated_residual
```

- static kernel specialization
 - Align input dimensions to the 2nd power
 - Ensure the kernel parameter BLOCK_SIZE is a compile-time constant to allow the compiler to perform full loop unrolling and eliminate runtime branch overhead.

Architectural Design of Operator Libraries

- Principles
- Dispatcher
- DSL
- Case Study



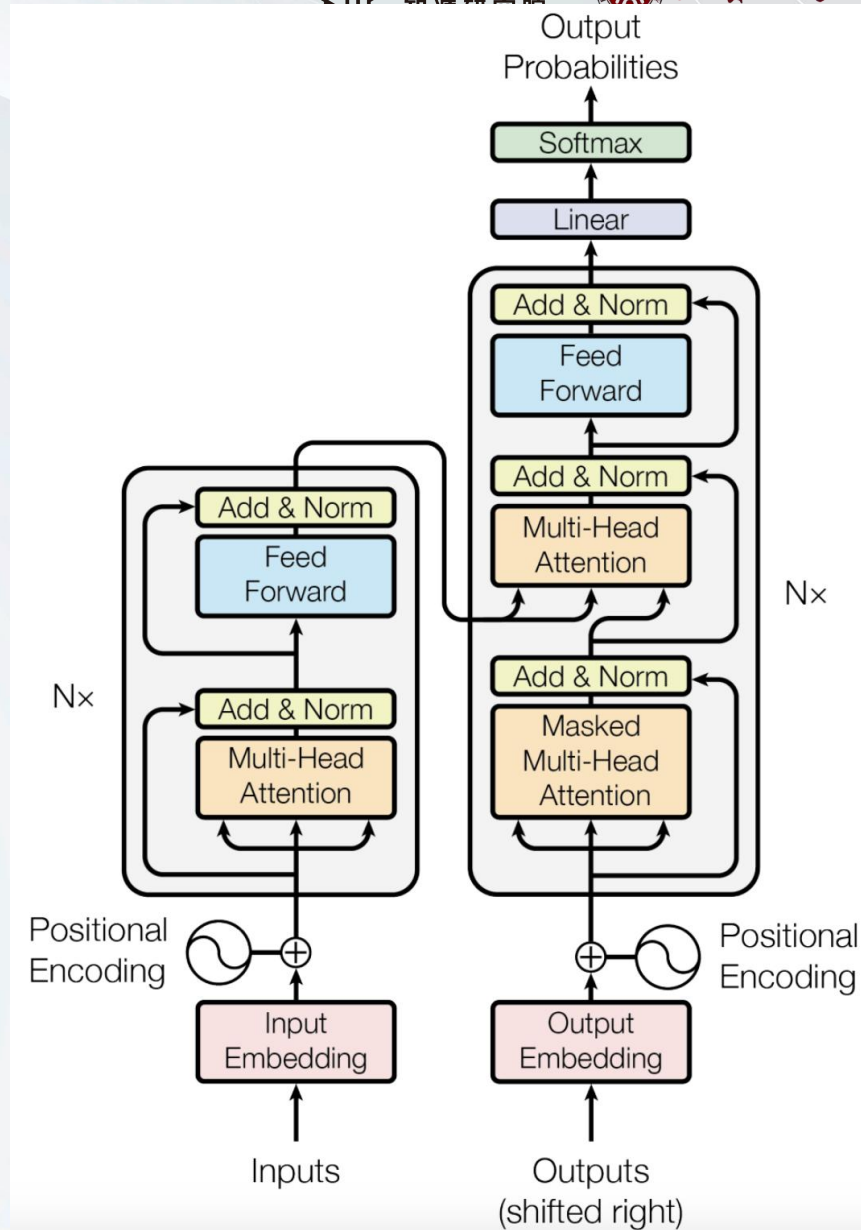
BAAI
智源研究院



北京大学
PEKING UNIVERSITY

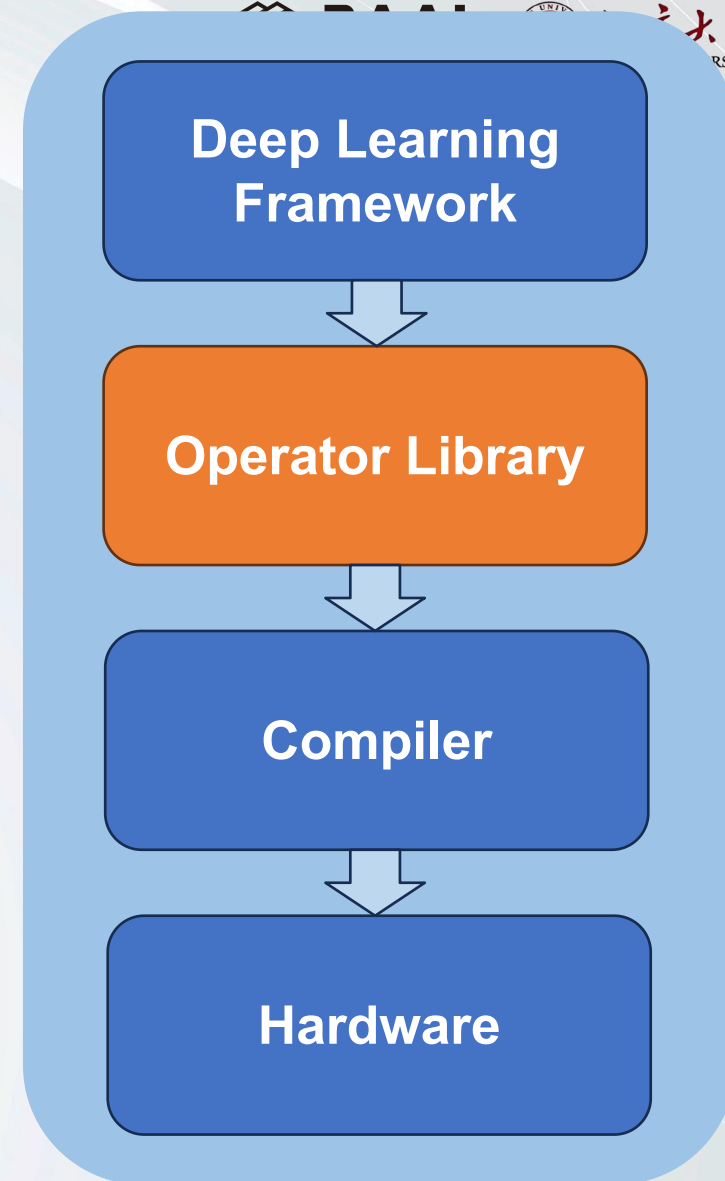
•Operator

- Broadly speaking, applying any specific operation to a given function can be defined as an Operator.
- Deep learning algorithms are constructed from individual computational units known as Operators.
- Examples: Add, MatMul, LayerNorm



• Operator Library

- A collection of highly optimized, reusable computational building blocks (Operators) for neural networks.
- Core Roles:
 - The Bridge - Connects high-level AI frameworks with low-level bare-metal hardware.
 - Performance Extractor - Maximizes hardware utilization via expert-level kernel tuning.
 - Hardware Abstraction - Shields algorithm developers from complex low-level programming.



• Design Principles - Usability



- Seamless Integration with High-Level Frameworks
 - Register the operator library directly into the framework's internal dispatch mechanism to allow existing models to benefit from acceleration with zero code modifications.
- Automatic Hardware Discovery
 - Implement runtime detection of the execution environment to automatically load the optimal backend without requiring manual user configuration.
- Uniform Interface for Diverse Backends
 - Define a standardized abstraction layer so that the same high-level API remains consistent across a wide variety of hardware architectures
- Multi-Level Abstraction Architecture
 - Offer a tiered abstraction system to meet diverse needs: high-level for rapid prototyping, mid-level for tuning parallelism and tiling, and low-level for expert optimization of hardware specifics like registers and warps

• Design Principles - Performance



- Persistent Caching and Pre-Tuning
 - Store optimized computational configurations and pre-warm kernels to eliminate initialization delays and ensure peak performance from the very first execution.
- Hardware-Aware Configuration Mapping
 - Utilize a configuration system that abstracts hardware-specific constraints, such as memory hierarchy and compute unit limits, into the code generation process.
- Adaptive Tiling
 - Decomposes operators into optimal compute tiles based on input dimensions, minimizing padding overhead and maximizing memory bandwidth utilization across variable sequence lengths.

• Trade-off

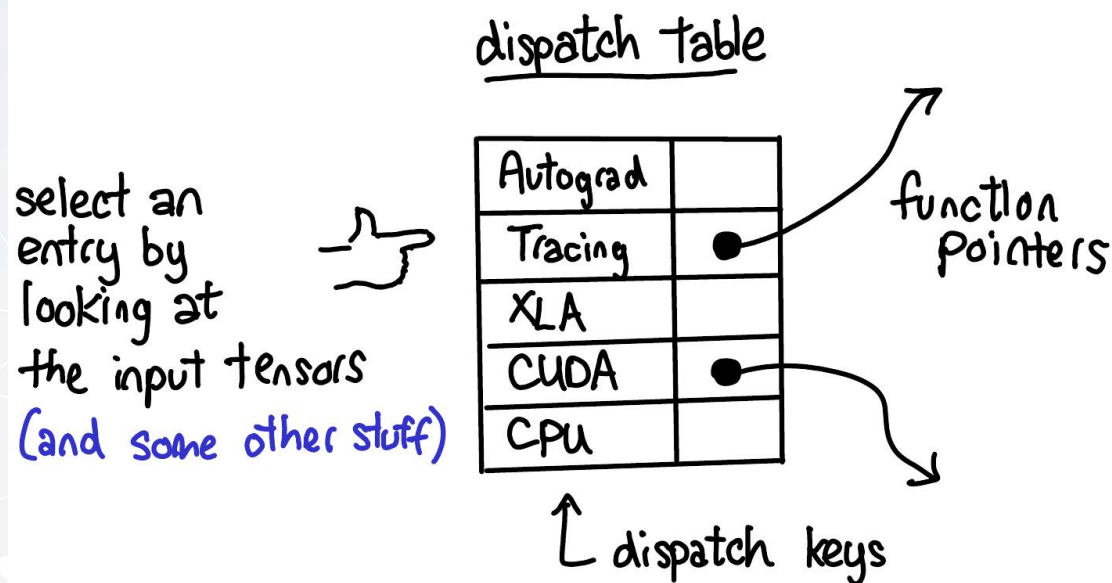


- Dynamic Shape vs. Hardware Alignment
 - Arbitrary input sizes often limit hardware-aligned peak performance
 - Solution: Adaptive Tiling
- Intuitive Layout vs. Optimal Memory Access
 - Standard data formats often conflict with hardware-optimized memory patterns
 - Solution: Layout Transformation
- Black-box API vs. Fine-grained Tuning
 - Simplified interfaces hide the tuning knobs needed for expert optimization
 - Solution: Heuristic Mapping
- A **dispatch mechanism** is required to manage these automated solutions and bridge the gap between user intent and hardware execution.

• Dispatcher

- A dispatcher is a component that **routes** an operation to the **appropriate implementation** or backend based on runtime criteria such as device type, data properties, or configuration policies.

Dispatcher: reimplemented vtables



differences from vtables:

- extensible at runtime → TLS
- multiple dispatch → boxing/unboxing

• Simplified Dispatch Workflow



- 1. Metadata Collection
 - Extract metadata such as device, data type, and layout from input tensors
- 2. Key Computation
 - Combine multiple attributes to generate a unique Dispatch Key
- 3. Table Lookup
 - Search the global Dispatch Table for the corresponding kernel function pointer
- 4. Execution
 - Invoke the specific backend implementation to perform the computation

• Advanced Dispatching Mechanisms



- Sequential Re-dispatching
 - A single operator call can trigger a chain of handlers by masking processed keys and re-invoking the dispatcher to find the next implementation.
- Heuristic Implementation Selection
 - Profile workload metrics (e.g., shapes) and hardware attributes (e.g., SM counts) to adaptively select optimal algorithm paths and execution parameters.
- Dynamic Kernel Instantiation
 - Specialized kernels can be generated and cached on-the-fly to handle various task dimensions or broadcasting patterns without manual coding.

• <https://blog.ezyang.com/2020/09/lets-talk-about-the-pytorch-dispatcher/>

• <https://github.com/flagos-ai/FlagGems>

• Use DSL to Implement Operator Libraries

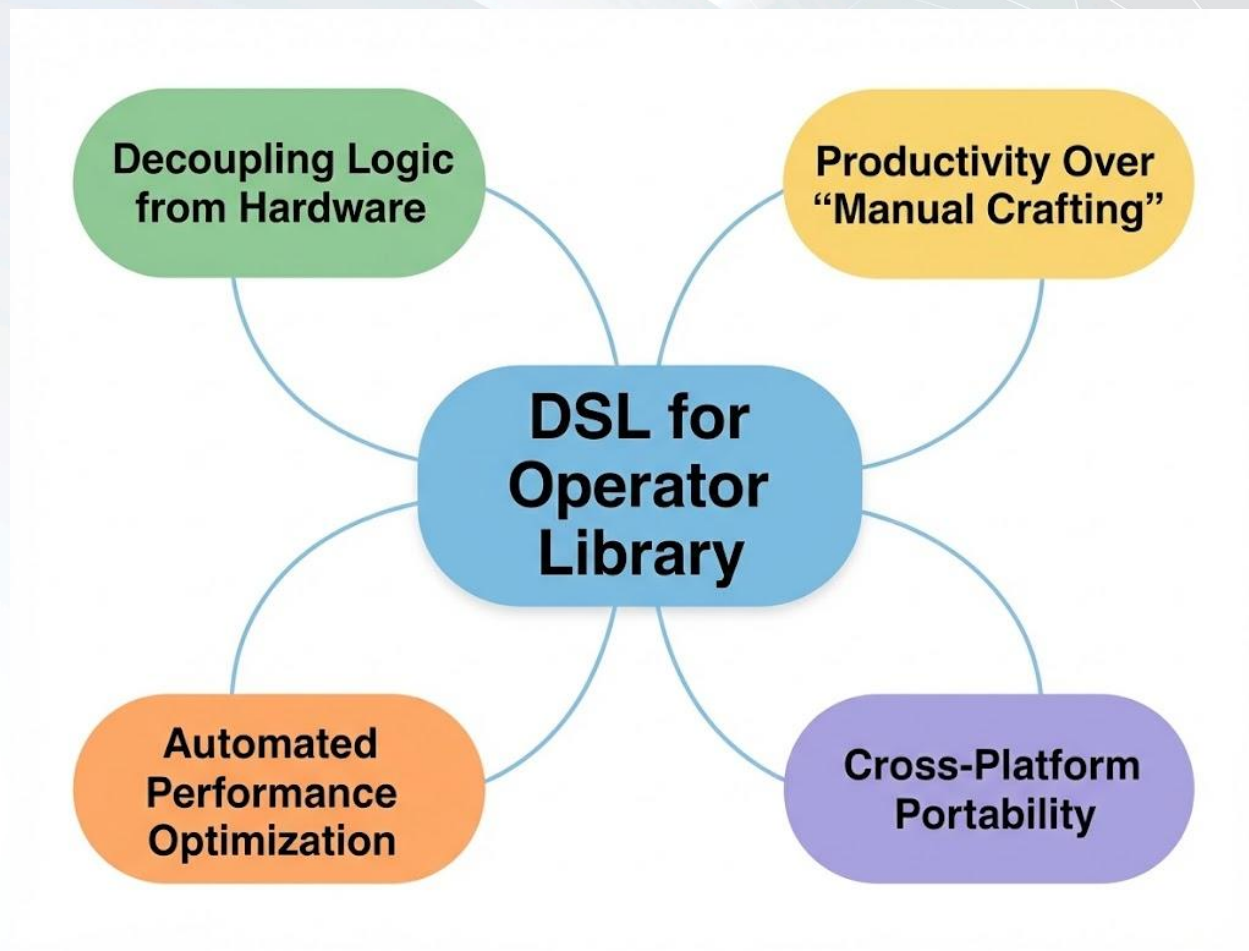


BAAI
智源研究院



北京大学
PEKING UNIVERSITY

- Developers describe “what to do,” while the compiler determines “how to do it.”
- A total separation between operator logic and physical implementation
- Core Objective: Make operator development simpler and more efficient while ensuring cross-platform portability
- Examples: Triton, TileLang, Gluon



• DSL for Operator Library - Triton



- Python-Based Tile Programming
- Compiler-Driven Optimization
- Hardware-Agnostic Abstraction



```
@triton.jit
def add_kernel(x_ptr, # *Pointer* to first input vector.
              y_ptr, # *Pointer* to second input vector.
              output_ptr, # *Pointer* to output vector.
              n_elements, # Size of the vector.
              BLOCK_SIZE: tl.constexpr, # Number of elements each program should process.
              # NOTE: `constexpr` so it can be used as a shape value.
              ):
    # There are multiple 'programs' processing different data. We identify which program
    # we are here:
    pid = tl.program_id(axis=0) # We use a 1D launch grid so axis is 0.
    # This program will process inputs that are offset from the initial data.
    # For instance, if you had a vector of length 256 and block_size of 64, the programs
    # would each access the elements [0:64, 64:128, 128:192, 192:256].
    # Note that offsets is a list of pointers:
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(0, BLOCK_SIZE)
    # Create a mask to guard memory operations against out-of-bounds accesses.
    mask = offsets < n_elements
    # Load x and y from DRAM, masking out any extra elements in case the input is not a
    # multiple of the block size.
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    output = x + y
    # Write x + y back to DRAM.
    tl.store(output_ptr + offsets, output, mask=mask)
```

• Case Study: Operator Definition



- Defining the Operator Schema
- Decoupling Logic from Hardware

```
# Keep these Library objects alive: registrations are released with the object.
_definition = torch.library.Library("lab", "DEF")
_definition.define(
    "fused_add_rms_norm(Tensor x, Tensor residual, Tensor weight, float eps=1e-6) "
    "-> (Tensor, Tensor)"
)

def fused_add_rms_norm_op(x, residual, weight, eps=1e-6):
    """Python convenience wrapper for ``torch.ops.lab.fused_add_rms_norm``."""
    return torch.ops.lab.fused_add_rms_norm(x, residual, weight, eps)
```

• Implementing the Dispatcher



• CUDA Dispatch - The Performance Path

```
def _cuda_impl(x, residual, weight, eps=1e-6):  
    """CUDA dispatch: invoke the raw Triton forward kernel."""  
    return fused_add_rms_norm(x, residual, weight, eps)  
  
# Register to CUDA Dispatch Key  
_cuda = torch.library.Library("lab", "IMPL", "CUDA")  
_cuda.impl("fused_add_rms_norm", _cuda_impl)
```

• Meta Dispatch - The Compilation Path

```
def _meta_impl(x, residual, weight, eps=1e-6):  
    """Fake/Meta dispatch used by Dynamo while it captures a compiled graph."""  
    # Return empty tensors with correct shapes (No actual computation)  
    return torch.empty_like(x), torch.empty_like(residual)  
  
# Register to Meta Dispatch Key  
_meta = torch.library.Library("lab", "IMPL", "Meta")  
_meta.impl("fused_add_rms_norm", _meta_impl)
```

• fused_add_rms_norm_backward_kernel



The screenshot displays the WorkBuddy interface with a search for "kernel" leading to an expert named "Multi-Chip Operator Development Expert". The expert's profile mentions 10+ years of LLM kernel development and proficiency in Triton. The interface shows a task description: "# Let z = x + residual and g = grad_out * weight. The gradient flowing through RMSNorm is: # grad_z_norm = g * rrms - z * rrms^3 * sum(g * z) / n_cols. The second returned forward value contributes grad_updated_residual directly to grad_z. Because z = x + residual, grad_x == grad_residual.## Accumulate grad_weight = grad_out * z * rrms across rows with # tl.atomic_add into the float32 grad_weight_ptr buffer. develop this operator using Triton". The expert has completed the task in 6 seconds. The resulting "KernelGen Operator Generation Report" specifies the operator as "rms_norm" (fused RMSNorm-with-residual, forward pass only), target device as "NVIDIA (CUDA)", and repository as "general Python/Triton workspace". The report also lists files written. Below the report, a chat input field is visible with the text "What can I help you with today? @ to reference files, / to invoke skills".

```
# Let z = x + residual and g = grad_out * weight. The gradient flowing
# through RMSNorm is:
# grad_z_norm = g * rrms - z * rrms^3 * sum(g * z) / n_cols
# The second returned forward value contributes grad_updated_residual
# directly to grad_z. Because z = x + residual, grad_x == grad_residual.
#
# Accumulate grad_weight = grad_out * z * rrms across rows with
# tl.atomic_add into the float32 grad_weight_ptr buffer.
row = tl.program_id(0)
cols = tl.arange(0, BLOCK_SIZE)
mask = cols < n_cols
offsets = row * n_cols + cols

z = tl.load(updated_residual_ptr + offsets, mask=mask, other=0.0).to(tl.float32)
weight = tl.load(weight_ptr + cols, mask=mask, other=0.0).to(tl.float32)
grad_out = tl.load(grad_out_ptr + offsets, mask=mask, other=0.0).to(tl.float32)
grad_z_direct = tl.load(
    grad_updated_residual_ptr + offsets, mask=mask, other=0.0
).to(tl.float32)

variance = tl.sum(z * z, axis=0) / n_cols
rrms = 1.0 / tl.sqrt(variance + eps)
scaled_grad = grad_out * weight
dot = tl.sum(scaled_grad * z, axis=0)

# d RMSNorm(z) = rrms * g - z * rrms^3 * sum(g * z) / N.
grad_z = (
    scaled_grad * rrms
    - z * (rrms * rrms * rrms) * dot / n_cols
    + grad_z_direct
)
tl.store(grad_x_ptr + offsets, grad_z, mask=mask)
tl.store(grad_residual_ptr + offsets, grad_z, mask=mask)

grad_weight = grad_out * z * rrms
tl.atomic_add(grad_weight_ptr + cols, grad_weight, mask=mask)
```

• FusedAddRMSNorm



```
class FusedAddRMSNorm(torch.autograd.Function):
    """Autograd wrapper around the forward and backward Triton kernels."""

    @staticmethod
    def forward(ctx, x, residual, weight, eps=1e-6):
        _check_inputs(x, residual, weight)
        out, updated_residual = fused_add_rms_norm(x, residual, weight, eps)
        ctx.save_for_backward(updated_residual, weight.contiguous())
        ctx.eps = eps
        return out, updated_residual

    @staticmethod
    def backward(ctx, grad_out, grad_updated_residual):
        updated_residual, weight = ctx.saved_tensors
        n_rows, n_cols = updated_residual.shape

        if grad_out is None:
            grad_out = torch.zeros_like(updated_residual)
        if grad_updated_residual is None:
            grad_updated_residual = torch.zeros_like(updated_residual)

        grad_x = torch.empty_like(updated_residual)
        grad_residual = torch.empty_like(updated_residual)
        grad_weight_fp32 = torch.zeros(
            (n_cols,), dtype=torch.float32, device=updated_residual.device
        )

        _fused_add_rms_norm_backward_kernel[(n_rows,)](
            updated_residual,
            weight,
            grad_out.contiguous(),
            grad_updated_residual.contiguous(),
            grad_x,
            grad_residual,
            grad_weight_fp32,
            n_cols,
            ctx.eps,
            BLOCK_SIZE=_block_size(n_cols),
        )

        return grad_x, grad_residual, grad_weight_fp32.to(weight.dtype), None
```

• Advanced Dispatching & Trade-offs

• Autograd Dispatch - Enabling Backward Pass

```
# Register to AutogradCUDA Dispatch Key
_autograd = torch.library.Library("lab", "IMPL", "AutogradCUDA")
_autograd.impl("fused_add_rms_norm", _autograd_impl)
```

• Sequential Re-dispatching & Trade-offs

```
def _autograd_impl(x, residual, weight, eps=1e-6):
    """Autograd dispatch."""
    # Trade-off: Handle FakeTensors during torch.compile tracing
    if isinstance(x, FakeTensor):
        return _meta_impl(x, residual, weight, eps) # Re-dispatch to Meta

    # Normal execution: Use Triton backward implementation
    return FusedAddRMSNorm.apply(x, residual, weight, eps)
```



• Thanks for your time!